



Rapport Final

UE PRO : Projet d'Etude

Drone : Stabilisation et évitement d'obstacles (PE n°80)

Elèves :

M. Baptiste GRIGNON

M. Thomas DESGREYS

M. Arthur SOUTELO ARAUJO

M. Paul MAESTRE

M. Mouhamadou KANE

M. Mattéo BEVILACQUA

Tuteur :

Dr. Charles-Edmond BICHOT

Conseiller Gestion de Projet :

M. Cédric GAMELON

Conseiller Communication :

M. Simon BACHELLIER

Version du
27 juillet 2026

Résumé

Les drones autonomes sont des systèmes amenés à réaliser des opérations variées sans intervention humaine. Ils doivent pour cela embarquer une large gamme de technologie assurant la robustesse du drone en vol. Ce rapport de Projet d'Étude porte sur les solutions existantes pour la réalisation des fonctions suivantes : stabilisation d'un drone, repérage de l'environnement pour l'évitement d'obstacles et navigation autonome du drone vers sa destination. Pour chaque fonction, différentes technologies de capteurs et de procédés sont documentées, et des solutions sont retenues pour une implémentation et une étude expérimentale en vue d'une application sur un drone quadrirotor. Le processus d'implémentation est détaillé puis les résultats des expériences sont présentés : détection d'obstacles avec des télémètres à ultrasons, reconnaissance d'image avec OpenCV et paramétrage du correcteur PID du contrôleur de vol Pixhawk 4 Mini.

Abstract

Autonomous UAVs are systems designed to achieve diverse tasks without the need of human input. For that purpose, a wide range of technologies is required onboard to ensure the robustness of the vehicle while in flight. This project report focuses on existing solutions in UAV technologies to achieve the following functions : stabilization of a vehicle, environment detection for obstacle avoidance and autonomous navigation towards the vehicle's objective. For each function, multiple technologies of sensors and processes are documented, then some of the solutions are selected for actual implementation on a quadrotor drone and further experimentation. The report presents the implementation process before presenting the results obtained for the following tests : obstacle detection with ultrasonic telemetry, image recognition with OpenCV and tuning of the PID controller on the PX4 flight controller.

Remerciements

Le groupe tient à exprimer ses sincères remerciements aux personnes qui l'ont aidé à mener à bien ce projet et à rédiger le rapport.

Nous tenons à remercier Dr. Charles-Edmond Bichot, notre tuteur, pour ses conseils, sa prévenance et sa disponibilité tout au long du projet. Nous lui sommes reconnaissant de l'aide et du temps qu'il nous a accordé.

Nous tenons ensuite à remercier M. Cédric Gamelon, notre conseiller en gestion de projet, pour son aide et ses conseils, notamment lors des rendez-vous de pilotage du projet, ainsi que M. Simon Bachellier, notre conseiller en communication pour son aide à la préparation de nos présentations et de nos rapports.

De plus, nous souhaitons remercier l'association Centrale Lyon Cosmos pour nous avoir confié ce projet et nous avoir accompagné dans sa réalisation.

Enfin, nous souhaitons également remercier les équipes du FabLab et du MakerLab pour leur aide tout au long de la réalisation de la maquette construite pour ce projet.

Table des matières

1	Introduction	2
1.1	Contexte et enjeux	2
1.2	Objectifs et démarche	2
1.3	Point de départ du projet	3
1.4	Tâches	4
2	Recherche de solutions	5
2.1	Stabilisation	5
2.1.1	Les technologies de capteurs	5
2.1.2	Les différentes stratégie de stabilisation	8
2.2	Repérage de l'environnement	9
2.2.1	Vision stéréoscopique	10
2.2.2	Lidar	12
2.2.3	Télémètres à ultrasons	13
2.2.4	Autres	15
2.2.5	Discussion	15
2.3	Reconnaissance d'image embarquée	16
2.3.1	Tensorflow	17
2.3.2	OpenCV	18
2.3.3	Keras	19
2.3.4	YOLO	20
2.3.5	Comparaisons	21
2.3.6	Choix du langage de programmation	21
2.4	Algorithmes de décision	23
2.4.1	Algorithme "Bug"	23
2.4.2	Méthodes de machine learning	24
2.4.3	Méthode du champ de potentiel virtuel	25
2.5	Bilan et choix	25
2.6	Budget et achat matériel	27
3	Implémentation et matériel	28
3.1	La stabilisation PID du PX4	28
3.1.1	Le pré-réglage	28
3.1.2	L'autotuning	28

3.1.3	Paramétrage manuel du PID	29
3.2	Utilisation de la RaspberryPi	30
3.2.1	Architecture pour l'évitement d'obstacles	30
3.2.2	Interfaces Capteurs - RaspberryPi	31
3.3	Capteurs pour l'évitement d'obstacles	32
3.3.1	Télémètres à ultrasons HC-SR04	32
3.3.2	Intel Realsense D420	34
3.4	Algorithme de reconnaissance d'image	35
3.4.1	Matériel utilisé	35
3.4.2	Algorithme et code python	36
4	Expériences et résultats	38
4.1	Télémètres à ultrasons	38
4.1.1	Premières mesures	38
4.1.2	Mesures de distances variables	39
4.1.3	Détermination de l'angle du cône de détection	40
4.2	PID	41
4.3	Reconnaissance d'image avec OpenCV	42
4.3.1	Reconnaissance de la cible	42
4.3.2	Positionnement de la cible	43
5	Conclusion	44
6	Annexes	48
6.1	Diagramme de GANTT	48
6.2	Tableau des tâches	49
6.3	Cahier des charges fonctionnel de l'étude	50
6.4	Connectique du drone : QGroundControl	51
6.5	Aide à l'utilisation de la RaspberryPi	52
6.5.1	Connexion à distance	52
6.5.2	Interfaces RaspberryPi - Pix Hawk 4 mini	53
6.6	Programmes de reconnaissance d'image	54
6.6.1	Script python implémentant la reconnaissance de la cible ainsi que son positionnement	54
6.6.2	Script Python permettant d'obtenir les paramètres propres à une couleur	56

6.7	Programmes de mesure de distance en utilisant les télémètres à ultrasons .	58
6.7.1	Mesure d'une distance par un capteur	58
6.7.2	Mesure de distance en continu	59
6.7.3	Code pour faire plusieurs mesures à distances croissantes	59
6.7.4	Code pour faire plusieurs mesures à distance constante et angle croissant	61
6.8	Check-list	64

Table des figures

1	Drone utilisé (photo issu du rapport de PE du groupe précédent)	4
2	Schéma de fonctionnement d'une caméra de profondeur [8]	11
3	Image de profondeur à droite et répartition en tâches à gauche [15]	12
4	Principe de la mesure LIDAR [4]	12
5	Principe de la mesure de distance par ultrasons	14
6	Score normalisé de consommation de différents langages [24]	22
7	Algorithme "Bug" de base [21]	23
8	Panel de réglage du contrôleur de vitesse	30
9	Schéma de l'architecture pour l'évitement d'obstacles	31
10	Disposition des pins GPIO de la RaspberryPi 4 B	31
11	Télémètres à ultrasons HC-SR04	32
12	Modèle pour la répartition des télémètres à ultrasons	33
13	Photographie du système de fixation des télémètres à ultrasons	34
14	Composants de la caméra de profondeur [16]	35
15	RaspberryPi équipée d'une caméra sur son port CSI	36
16	Exemple de paramétrage de la couleur rouge bordeaux	37
17	Mesure de distance en continu	39
18	Résultats des mesures de distance effectué avec le capteur à ultrasons	40
19	Résultats des mesures de distance en fonction de l'angle entre l'axe du capteur et la droite capteur-obstacle	41
20	Résultat de l'expérience de la reconnaissance de couleur et de forme (seul le carré rouge est détecté)	42
21	Résultat de l'expérience de positionnement de la cible	43
22	Diagramme GANTT (20/03/2022)	48
23	Cahier des charges fonctionnel de l'étude	50
24	QGround	51
25	Interface de QGroundControl	51

Liste des tableaux

1	Répartition des tâches	4
2	Budget	27
3	Avancement des tâches	49

Glossaire

Télémètre : Instrument d'optique servant à mesurer la distance qui sépare un observateur d'un point éloigné.

Reconnaissance d'image : ensembles de procédés par lesquels une intelligence artificielle est capable de détecter la présence d'un objet sur une image numérique.

Bibliothèque (logicielle) : collection de **routines** pré-compilées et prêtes à être utilisées par des programmes.

Routine : ensemble d'instructions qui prend en charge une certaine opération et produit un résultat. Elles sont utilisées pour les opérations fréquentes de programmation.

Réseau neuronal convolutif : type de réseau de neurones artificiels acycliques, dans lequel le motif de connexion entre les neurones est inspiré par le cortex visuel des animaux.

Contrôleur de vol : ordinateur de bord du drone mettant en relation tous les organes du drone et commandant les moteurs.

1 Introduction

1.1 Contexte et enjeux

Ce Projet d'Etude n°80 fait suite au PE n°63 [28] de l'année 2020-2021 portant sur la conception et la réalisation d'un drone autonome.

La navigation autonome est un enjeu crucial pour la robotisation car elle demande une qualité jusqu'ici réservée aux êtres vivants : l'adaptabilité à des situations variables. En effet, dans des cas d'utilisation où les conditions d'usage ne sont pas clairement définies, l'être humain sera toujours plus efficace qu'une machine. Par exemple, un robot sur une chaîne de montage sera efficace seulement lorsque les bonnes pièces lui parviennent. Un opérateur reste nécessaire si une pièce défectueuse se casse dans la chaîne de montage pour remettre le système dans ses conditions normales. Or, dans certains domaines d'application les conditions sont beaucoup plus variables et le système doit pouvoir réagir en temps réel. C'est à cette problématique que ce projet s'intéresse, dans le cas d'un drone autonome.

Les domaines d'application d'un tel drone sont multiples comme la livraison de colis, la surveillance de sites sensibles, ou encore l'exploration de lieux impossibles d'accès pour l'être humain. La commande à distance n'est pas toujours envisageable, comme dans un site isolé des ondes électromagnétiques ou dans le cas où le nombre de drones à commander est trop important, comme pour un système de livraison de colis. Dans ces cas, chaque drone doit pouvoir identifier dans son environnement ses objectifs, comme une zone de dépôt, ainsi que les obstacles à éviter comme un muret, afin d'accomplir sa mission sans embûches.

Ce projet portera donc sur l'étude d'un drone autonome et stable.

Le projet est commandité par l'association Centrale Lyon Cosmos qui propose des Projets d'Etude dans le domaine de l'aérospatial et qui a initié le projet de l'année précédente. Le PE n°80 est encadré par Dr. Bichot (Tuteur), M. Gamelon (Conseiller en Gestion de Projet) et M. Bachellier (Conseiller en Communication).

1.2 Objectifs et démarche

L'objectif du PE n°80 est de faire avancer les connaissances en matière de drone, et ce, sur les axes suivants :

- La stabilisation des drones ;
- La navigation autonome et l'évitement automatique des obstacles.

L'étude porte notamment sur des méthodes de reconnaissance d'image embarquée pour la navigation du drone car il s'agit de la solution retenue par le PE précédent.

La démarche adoptée pour ce projet d'étude commence par des recherches documentaires des solutions :

- de stabilisation des drones ;
- d'évitement d'obstacles ;
- de reconnaissance d'image embarquée.

Ces recherches seront ensuite synthétisée dans le présent rapport. Sur la base des solutions documentées, certaines seront implémentées et étudiées expérimentalement. Le cahier des charges de l'étude est disponible en annexe 23

Les livrables comprennent donc le drone et ses systèmes attendant, les maquettes de tests réalisées, le code informatique développé et le présent rapport détaillant les solutions étudiées, leur fonctionnement et les résultats des essais.

1.3 Point de départ du projet

Le point de départ du projet a été le travail effectué par le groupe du projet d'étude précédent et notamment le matériel acheté.

Le drone quadrirotor hérité du PE précédent 1 est composé d'un kit de base du fabricant Holybro (Basic Kit QAV250) contenant un châssis en fibres de carbones, le contrôleur de vol PixHawk 4 mini, de 4 moteurs DR2205 de Kv2300, des hélices en plastique, un module GPS Pixhawk 4, une platine de gestion d'alimentation, des câbles d'alimentation, et un transmetteur radio. Le drone est également équipé de batteries LiPo et accompagné d'une télécommande. Un capteur PIX-FLOW permettant l'estimation de la position du drone n'est pas implémenté.

Le groupe précédent a également acheté une RaspberryPi 4 Modèle B 4Go ainsi qu'une caméra Arducam 5MP pour RaspberryPi.



FIGURE 1 – Drone utilisé (photo issu du rapport de PE du groupe précédent)

1.4 Tâches

Pour ce projet, le groupe a décidé de s'organiser en 3 pôles explicités dans le tableau 1.

Stabilisation	Évitement d'obstacle	Reconnaissance d'image embarquée
• Paul MAESTRE (Responsable) • Arthur SOUTELO ARAUJO • Mouhamadou KANE	• Baptiste GRIGNON (Perception de l'environnement, Responsable) • Mattéo BEVILACQUA (Algorithmes de décisions/planification trajectoire) • Arthur SOUTELO ARAUJO	• Thomas DESGREYS (Responsable, Chef de Projet) • Mouhamadou KANE

TABLE 1 – Répartition des tâches

Le détail des tâches du projet et de leur avancement est également disponible en annexe : tableau 3 et diagramme de GANTT 22.

2 Recherche de solutions

Cette première partie a pour but de faire un état de l'art des solutions sur les points abordés dans l'étude à partir de recherches documentaires. Il s'agit de ne pas foncer tête baissée dans une solution à cause d'une idée préconçue. Les recherches s'orientent selon 4 axes :

- les techniques de **stabilisation de drones** ;
- les techniques permettant le **repérage de l'environnement**, c'est à dire l'acquisition d'informations exploitables sur son milieu extérieur ;
- les **stratégies et algorithmes de traitement** de ces informations pour l'évitement d'obstacles ;
- les techniques de **reconnaissance d'image** dans un flux vidéo sur micro-ordinateur embarqué.

La grande majorité des articles est en anglais donc les bons mots clés doivent être utilisés pour les recherches, au risque de perdre un temps certain à lire des articles qui ne sont pas pertinents. En voici quelques uns :

- *UAV* pour *Unmanned Aerial Vehicle* ce qui désigne la même chose que le terme *drone* en français ;
- *MAV* pour *Micro Aerial Vehicle* qui désigne les drones de petite taille ;
- *Obstacle Avoidance* pour évitement d'obstacle ;
- *Computer Vision* pour vision par ordinateur, qui englobe les différentes techniques de reconnaissance et de traitement d'image ;
- *Linear/Nonlinear Control* pour commande linéaire/non linéaire.

2.1 Stabilisation

2.1.1 Les technologies de capteurs

Pour la mise en oeuvre des méthodes de stabilisation, des capteurs sont nécessaires pour obtenir des informations sur la position du drone et réaliser une boucle de régulation. Nos recherches sur les différentes technologies de capteurs se basent principalement sur les travaux de thèse d'Adrien DROUOT [11].

a) Accéléromètre

L'accéléromètre mesure l'accélération du drone selon un axe de l'espace. Un accéléromètre 3D mesure l'accélération du drone dans l'espace, donc selon 3 axes.

b) Gyroscope

Le gyroscope permet de mesurer la variation de la position angulaire du drone autour d'un axe. Un gyroscope dit "3-axes" mesure l'angle selon les 3 axes du drone :

- Rouleau : rotation autour de l'axe avant-arrière
- Pas : rotation autour de l'axe latéral
- Lacet : rotation autour de l'axe vertical

c) Localisation GPS

La localisation GPS permet de déterminer la latitude, la longitude et l'altitude du drone avec une précision de l'ordre de la dizaine de mètres. La localisation GPS fonctionne par émission et réception de signaux entre le capteur et plusieurs satellites en orbite et calcul de la distance en fonction du délai entre émission et réception.

d) Télémétrie

La télémétrie permet de déterminer la position du drone à partir de son environnement. Les capteurs à télémétrie consistent en des systèmes émetteurs/récepteurs : en émettant un signal et en recevant sa réflexion dans l'environnement, on peut mesurer la distance entre le drone et les éléments physiques de son entourage selon l'intervalle entre émission et réception. Le type d'onde utilisé dépend du capteur mis en œuvre :

- Ondes acoustiques ou SONAR (de l'anglais *SOund Navigation And Ranging*), ou encore SODAR (de l'anglais *SOnic Detection And Ranging*).
- Ondes optiques ou LIDAR (de l'anglais *LIght Detection And Ranging*) : les ondes électromagnétiques utilisées peuvent être du domaine ultraviolet, visible ou bien infrarouge.
- Ondes radioélectriques, ou RADAR (de l'anglais *RAdio Detection And Ranging*), capables également de déterminer la vitesse des objets détectés par effet Doppler.

e) Baromètre

Un altimètre barométrique permet d'avoir une information sur l'altitude du drone par mesure de la pression atmosphérique. Pour des vols en extérieur, cette mesure est cependant fortement influencée par les conditions météorologiques.

f) Compas (boussole)

Le compas permet de connaître l'orientation absolue du drone selon l'axe vertical, généralement par rapport à la direction du nord magnétique. Il existe de nombreuses technologies de compas, dont les magnétomètres utilisant un aimant, les compas électroniques utilisant des matériaux sensibles au champ magnétique ou encore les compas gyroscopique qui utilisent un gyroscope aligné au préalable sur la direction du nord.

g) Caméras

Des caméras peuvent être également utilisées pour la stabilisation du drone. Ces méthodes impliquent de nouvelles contraintes de luminosité et nécessitent une méthode de traitement des images pour en extraire l'information utile.

Il est possible d'utiliser le flux optique d'une caméra pour déterminer le déplacement du drone par rapport à l'environnement. Cette utilisation est possible dans les phases de décollage et d'atterrissage, ou bien pour la stabilisation en vol.

L'utilisation de deux caméras au moins permet d'obtenir également une information sur la profondeur des images et de cartographier l'environnement. On peut ainsi estimer la position et l'orientation du drone dans l'espace.

h) Centrale inertielle

Une centrale inertielle (ou *Inertial Measurement Unit (IMU)* en anglais) se compose de trois accéléromètres et de trois gyroscopes, couplés afin de déterminer l'accélération non gravitationnelle du drone et sa vitesse de rotation. Par intégration, on accède à la position et à l'orientation du drone par rapport à son initialisation. Pour minimiser les erreurs dérivant des intégrations des mesures, on peut utiliser un algorithme de fusion de données. Par exemple, le filtre de Kalman permet de déterminer les erreurs des gyroscopes et accéléromètres selon les données GPS.

i) Capteurs inclus sur le PixHawk 4

Le contrôleur de vol PixHawk 4 mini [3] installé sur notre drone inclut certains des capteurs présentés précédemment :

- Deux centrales inertielles : ICM-20689 et BMI055
- Un magnétomètre : IST8310
- Un baromètre : MS5611
- Un capteur GPS : u-blox Neo-M8 avec récepteur GPS/GLONASS, et magnétomètre IST8310 intégré

Le minimum requis pour le vol du PixHawk 4 est le gyroscope, l'accéléromètre, le compas et le baromètre. Pour des phases de vol autonome, le capteur GPS est aussi nécessaire.

2.1.2 Les différentes stratégie de stabilisation

La stabilisation se fait grâce à des stratégies de commande. Ces dernières sont soit linéaires, soit non linéaires. Les systèmes de stabilisation non linéaires sont plus complexes mais peuvent offrir une meilleure précision de stabilisation. Dans le cadre de notre projet, nous nous concentrons sur des stratégies de commande linéaires qui sont plus simple à implémenter et qui suffisent dans la plupart des situations de stabilisation de drones quadricoptère. La mise en place de ces stratégies de stabilisation commence par la modélisation mathématique des effets physiques affectant le vol du drone. Les informations ci-dessous sont en grande partie inspirées de la thèse d'Adrien DROUOT [11].

a) Commande par synthèse $\mathcal{H}_\infty$:

La synthèse $\mathcal{H}_\infty$ est une méthode de conception d'algorithmes de commande dits optimaux, garantissant théoriquement un certain niveau de performance et de robustesse. L'objectif central de cette approche est de minimiser la norme $\mathcal{H}_\infty$ d'un système dynamique. En effet, cette norme caractérise l'amplification maximale que peut appliquer le système sur le signal d'entrée ce qui, dans le cas d'un système SISO (une seule entrée et une seule sortie), se traduit par la valeur maximale de l'amplitude de sa réponse fréquentielle. Il existe plusieurs méthodes permettant le calcul et l'optimisation de la norme $\mathcal{H}_\infty$:

- méthode basée sur la résolution des équations de Riccati
- méthode(s) basée sur les Inégalités matricielles linéaires
- méthode utilisant la paramétrisation de Youla

b) Commande par placement de pôles :

La commande par placement de pôles, également appelée commande modale, est une méthode permettant de piloter un système linéaire à l'aide d'une combinaison linéaire des états du système. Elle n'est donc réalisable que si tout le vecteur d'état est accessible à la mesure. Cette approche permet de fixer précisément les pôles de la fonction de transfert en boucle fermée du système dans le but d'obtenir un comportement dynamique approprié.

L'objectif de cette approche est ainsi de garantir que le système bouclé adopte un comportement convenable par un choix judicieux des valeurs propres de sa matrice d'évolution. En effet, la position de ces dernières est étroitement liée au comportement temporel du système, notamment en termes de stabilité, de rapidité et d'amortissement.

c) Commande par régulation PID :

La commande par régulation PID (pour Proportionnelle - Intégrale - Dérivée) est certainement la structure de commande la plus couramment utilisée dans l'industrie. La prévalence de cette approche vient, au-delà de sa simplicité, des performances qu'elle offre aux systèmes pilotés en boucle fermée, quel que soit leur domaine d'application. Le signal de commande est la somme de trois termes : un terme proportionnel à l'erreur, un terme proportionnel à l'intégrale de l'erreur et un terme proportionnel à la dérivée de l'erreur. Les parties intégrale, proportionnelle et dérivée pouvant être considérées comme des actions de commande basées sur le passé, le présent et le futur respectivement. L'adaptation de la commande aux exigences du système en boucle fermée se fait en paramétrant de façon appropriée le gain proportionnel K_p , le gain intégral K_i et le gain dérivé K_d .

2.2 Repérage de l'environnement

L'évitement d'obstacle par un drone peut se décomposer en 3 étapes :

- l'acquisition des informations sur l'environnement, traitée dans cette partie
- la prise de décision et la planification de la trajectoire à partir des informations collectées
- la traduction de cette prise de décision en ordres pour le drone et leur exécution par celui-ci

Le but est de réaliser ces 3 étapes en boucle avec la plus grande fréquence possible afin de s'adapter en temps réel aux changements de l'environnement et au mouvement même du drone en fonction des obstacles qui entrent ou sortent de la portée des capteurs.

L'évitement d'obstacle s'ajoute à la mission principale du drone : se déplacer d'un point A à un point B, et ne doit pas la gêner. La planification de la trajectoire doit se faire par rapport au point d'arrivée et sera traitée dans une partie suivante.

L'objectif vers lequel se diriger peut être un point de l'espace défini arbitrairement ou bien un objet dans l'environnement du drone. L'identification de cet objet peut passer par de la reconnaissance d'image embarquée sur le drone et sera l'objet d'une partie suivante.

La vision de l'évitement d'obstacle utilisée ci-dessus est inspirée par les travaux de BONNIFAIT et ZINOONE [8] sur les techniques de navigation des véhicules autonomes. Aussi, ils introduisent des méthodes de perception de l'environnement utilisés sur les véhicules autonomes terrestres, dont nous pouvons nous inspirer pour les drones :

- caméras matricielles "classiques" (capteur CMOS, flux RGB) dont la vidéo est ensuite traitée.
- capteurs radars
- capteurs à ultrasons
- capteurs laser à balayage (LIDAR)
- caméras non-conventionnelles : thermiques, à évènements ...

Cette partie traitera des méthodes de perception de l'environnement applicables aux drones.

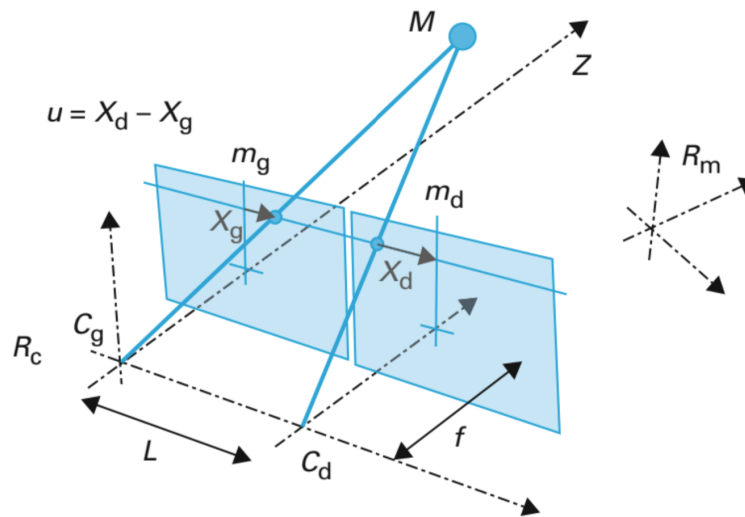
2.2.1 Vision stéréoscopique

La vision stéréoscopique repose sur l'utilisation de 2 caméras pour recomposer les informations de profondeur d'une image, à l'instar des yeux humains. C'est l'une des techniques sur lesquelles la recherche se concentre pour réaliser l'acquisition des informations sur l'environnement par des drones car elle fournit des informations précises à une portée raisonnable sur un champ de vision très large devant le drone.

Une caméra de profondeur utilise deux caméras afin d'obtenir un flux vidéo "3D" : le flux vidéo 2D et la profondeur des éléments dans le champ de vision. Pour cela, les 2 caméras sont solidairement positionnées (à une distance connue) et la mesure de la position relative de l'objet par rapport aux deux caméras permet de déduire sa profondeur (effet parallaxe).

Une caméra de profondeur est donc le système comportant les 2 caméras et un processeur pour calculer la profondeur par triangulation.

Les travaux de LE BESNERAIS [6] explique comment retrouver la profondeur d'un point à partir des images de 2 caméras :



Les deux caméras C_g et C_d ont des plans images confondus, leurs axes optiques sont parallèles, elles sont séparées de la distance L appelée ligne de base. Les points homologues m_g et m_d sont situés sur la même ligne dans les deux images, leur différence de position dans le repère image est appelée disparité, notée u .

FIGURE 2 – Schéma de fonctionnement d'une caméra de profondeur [8]

On a alors la relation simple donnant la profondeur du point M : $Z = \frac{f \times L}{u}$ avec f la focale (commune) des deux caméras. Ce modèle nécessite de connaître 2 points homologues, c'est à dire qui représentent le même objet réel sur les 2 caméras. Les méthodes pour déterminer les points homologues sont diverses, le rôle du processeur qui accompagne les 2 caméras est d'exécuter ces méthodes.

Une fois les données de profondeur obtenues, leur exploitation dépend de l'approche prise dans la stratégie plus globale d'évitement d'obstacle. Par exemple, dans les travaux de BRUNNER [9], le micro-ordinateur embarqué construit une grille d'occupation : l'espace est découpé en cases et utilise les informations de profondeur pour déterminer quelles cases de l'espace sont occupées par un obstacle et dans lesquelles le drone peut naviguer. Dans ses travaux, HU [15] utilise une technique de vision par ordinateur, l'analyse de taches appliquée à la "couche" de l'image correspondant correspondant à une distance entre 0.5 m et 2 m du drone (il ne s'intéresse pas aux obstacles à plus de 2m, et se pose s'il rencontre un obstacle à moins de 50 cm). Cette technique identifie des taches de couleurs qui ici correspondent à des obstacles dont la couleur (en nuance de gris) est l'indication de la distance, comme sur la figure 3. La position du centroïde des taches dans l'image et sa

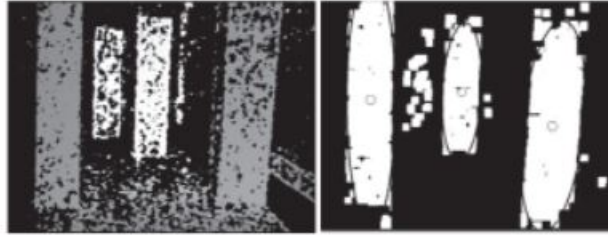


FIGURE 3 – Image de profondeur à droite et répartition en taches à gauche [15]

couleur donne ainsi une indication sur la position relative d'un obstacle par rapport au drone. Un traitement plus simple est aussi possible. En effet, dans ses travaux, YANG [33] considère seulement la zone au centre de l'image, c'est à dire là où se dirige le drone, pour l'évitement d'obstacle.

Il est important de noter qu'en dessous d'une certaine distance caméra - obstacle, l'évaluation de profondeur est faussée et la distance est considérée nulle.

2.2.2 Lidar

La technologie LIDAR (pour *Light Detection And Ranging*) est aussi très utilisée pour l'évitement d'obstacle pour le drone. Le principe est d'utiliser un laser pour déterminer la distance en face du capteur. Ce type de mesure possède une grande portée et une bonne précision, et peut être répétée à grande fréquence. Le principe de la mesure est le suivant :

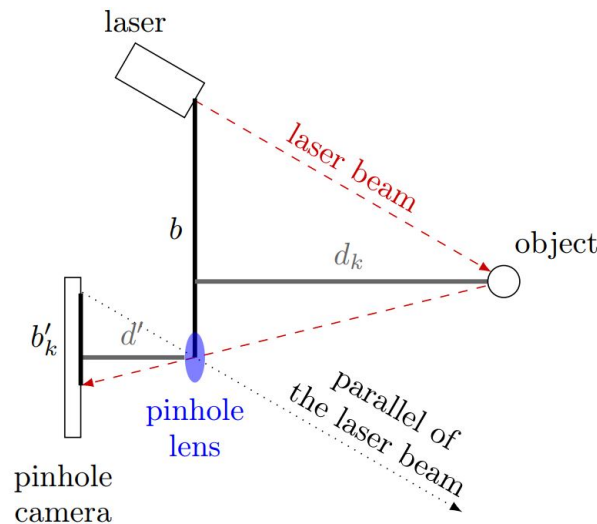


FIGURE 4 – Principe de la mesure LIDAR [4]

Les relations de trigonométrie permettent de déduire

$$d_k = \frac{d'b}{b'_k}$$

La mesure concrète dépend de la technologie des capteurs photosensibles utilisés pour la caméra. La figure et le principe sont extraits des travaux de ALHASHIMI [4].

Un capteur LIDAR en tant que tel permet seulement d'avoir une mesure de distance dans une seule direction. Une manière courante de l'utiliser est de balayer l'environnement d'un faisceau LIDAR. Dans ses travaux NIKKO [10] motorise un tel capteur LIDAR à simple faisceau afin de lui faire balayer à 360° dans un plan horizontal. Une variation de ce système est un capteur LIDAR qui inclue directement le balayage à l'aide d'un système de miroir motorisé. Ce type de système est bien plus cher qu'un capteur LIDAR simple, et il n'a pas un angle de perception de 360° à cause de contraintes dues à la conception du système, mais il permet d'avoir un balayage de vitesse angulaire beaucoup plus rapide. FERRICK [12] l'utilise dans ses travaux. Il combine les points obtenus par la mesure LIDAR avec des techniques de traitement de l'image pour obtenir les zones sûres de circulation du drone et lui faire suivre les murs. Une autre façon d'exploiter les mesures d'un capteur LIDAR à balayage est via l'utilisation d'un algorithme SLAM comme le fait XIN dans ses travaux [31]. SLAM signifie *Simultaneous Localization And Mapping*, ce type d'algorithme permet à partir des points mesurés par le capteur LIDAR de déterminer à la fois une carte de l'environnement et la position du drone dans celle-ci.

2.2.3 Télémètres à ultrasons

Les télémètres à ultrasons sont la solution la moins chère et la plus simple pour détecter un obstacle. Une impulsion ultrasonore est envoyée (figure 5) et réfléchiée sur un obstacle.

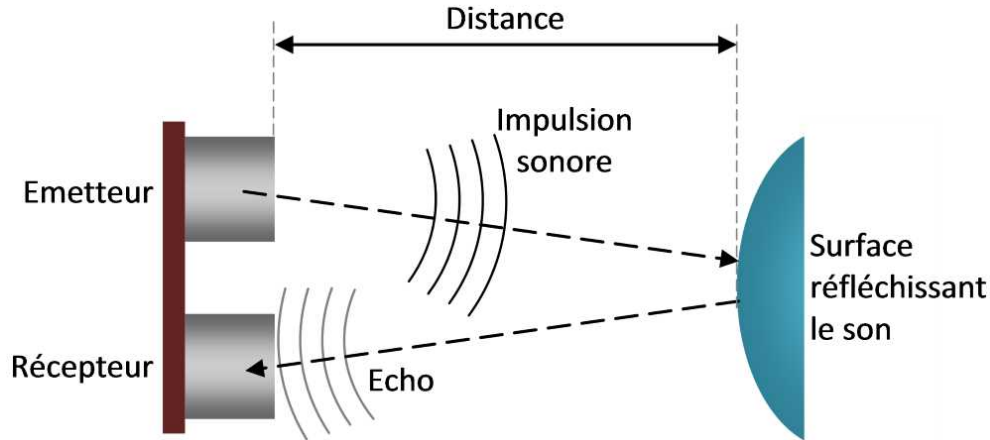


FIGURE 5 – Principe de la mesure de distance par ultrasons

La durée entre l'émission et la réception du signal Δt est l'image de la distance D :

$$D = c * \frac{\Delta t}{2}$$

avec c la célérité du son dans l'air. Lorsque la durée sans écho devient plus grande que la durée correspondant à un obstacle en limite de portée du capteur, il est possible d'arrêter la mesure et de considérer qu'il n'y a pas d'obstacle devant le capteur.

Cette méthode est à faible portée et est peu précise car les ultrasons se propagent dans un cône plus ou moins large mais permet d'obtenir facilement des informations de distance à un obstacle dans une direction. Il est possible d'utiliser plusieurs capteurs pour couvrir plusieurs directions. La version la plus élémentaire de cette solution utilise 4 capteurs pour obtenir les informations devant, derrière et sur les côtés, comme dans les travaux de XU [32]. Cette méthode présente encore beaucoup d'angles morts et NIWA [18] propose une méthode de détection d'obstacles utilisant 8 capteurs ce qui réduit de beaucoup les angles morts. Dans ces travaux, les capteurs ne sont pas disposés sur les côtés d'un octogone régulier mais associés 2 à 2 ce qui permet de choisir des directions préférentielles de détection. GUANGLEI [13] propose une méthode utilisant seulement 4 capteurs, mais dans laquelle chaque capteur est capable de détecter les impulsions émises par les autres capteurs, et de les identifier ce qui permet d'estimer la position des obstacles qui ne sont pas directement en face d'un capteur.

2.2.4 Autres

D'autres solutions sont également possibles, comme des RADARs spécialement conçus pour l'évitement d'obstacles par des drones. Ainsi, SONG [25] propose une architecture pour une antenne produisant un faisceau électromagnétique étroit de fréquence 77GHz pouvant être utilisés par des drones pour estimer la distance à des obstacles.

2.2.5 Discussion

Le but de cette partie est de comparer, quantitativement si possible, les différentes solutions pour le repérage de l'environnement pour l'évitement d'obstacle. Selon les modèles, les performances peuvent beaucoup varier. Les valeurs ci-après sont des ordres de grandeur.

Portée :

- Caméra de profondeur : 12m;
- Télémètres à ultrasons : 3m;
- LIDAR : 25m.

Le LIDAR a les meilleures performances sur ce point mais on peut se demander si 25m de portée de détection sont nécessaires pour un drone. Cela va dépendre de la vitesse de celui-ci mais une dizaine de mètres devrait être amplement suffisant, voire 3-4 mètres s'il va lentement.

Champs de vision :

Cela dépend de la technologie précisément utilisée mais la caméra de profondeur aura le meilleur champ de vision car il peut valoir jusqu'à 90° à l'horizontale et 60° à la verticale. En comparaison, le LIDAR mesure un unique point, ou dans le cas d'un LIDAR à balayage dans un plan sur un angle allant jusqu'à 270°, mais il ne permet pas d'obtenir des informations en dessous ou au-dessus de ce point. Le capteur à ultrasons détecte les obstacles dans un cône d'environ 30° dans une direction.

Encombrement et conditions d'utilisation :

Pour une détection d'obstacles efficace avec des capteurs à ultrasons, il faut en mettre plusieurs ce qui alourdit le drone et prend de la place, cependant la puissance de calcul nécessaire pour ces mesures est très faible comparée à celle nécessaire pour l'utilisation d'une caméra de profondeur (pour le LIDAR, la puissance de calcul nécessaire dépend

beaucoup de l'application). La caméra de profondeur et un LIDAR à balayage créent un encombrement spatiale moins important que plusieurs télémètres à ultrasons.

La caméra de profondeur est une technologie passive, c'est-à-dire qu'elle utilise la lumière ambiante. Elle sera donc peu fiable dans un environnement obscur ou nocturne, contrairement au LIDAR et au télémètre à ultrasons qui génèrent leur propre moyens de mesures.

Coûts :

- Caméra de profondeur : 140€ minimum pour les pièces détachées (carte de calcul + caméras en elle même), à partir de 300€ pour une caméra de profondeur prête "sur étagère".
- LIDAR : 150€ pour un LIDAR unidirectionnel et plus de 500€ pour un LIDAR à balayage. Ces valeurs dépendent énormément des performances comme la portée, la résolution, la consommation électrique et la fréquence de balayage pour les LIDAR à balayage. En particulier, certains modèles utilisent des LEDs et non pas des lasers ce qui diminue les coûts.
- Télémètre à ultrasons : 3€. Incontestablement le moins coûteux.

2.3 Reconnaissance d'image embarquée

Dans la perspective de pouvoir identifier les objets dans l'environnement du drone, cette partie traitera les techniques de reconnaissance d'images. Il est important de noter que la reconnaissance d'image seule (sur un flux vidéo RGB) ne permet pas de localiser les objets reconnus dans l'espace (pour le cas général, des techniques particulières existent, exemple : connaître la taille réelle de l'objet et utiliser la taille relative disponible sur le flux vidéo pour estimer la distance). La reconnaissance d'image doit être associée aux informations données par les autres capteurs afin de localiser l'objet identifié dans l'espace.

Pour traiter ces informations, nous utiliserons des outils de d'analyse d'image et de reconnaissance d'objets, avec entraînement (algorithmes de *deep learning*) ou pré-enregistrés (bibliothèques). Pour choisir parmi les outils existants, nous pourrons les discriminer en fonction de leur vitesse de reconnaissance d'image (puisque l'on veut une reconnaissance d'image en temps réel) et leur consommation en ressources (en calculs et en énergie). Enfin, de manière plus pragmatique, on s'intéressera à leur simplicité d'implémentation, de la qualité de la documentation disponible, de leur compatibilité avec les cartes électroniques

(Raspberry, Arduino, ...) et les langages de programmation à notre disposition.

Nous présenterons successivement les technologies suivantes : Tensorflow, OpenCV, Keras, et Yolo. Nous procéderons ensuite à une comparaison de ces dernières.

2.3.1 Tensorflow

TensorFlow est un *framework* de programmation Open Source pour le calcul numérique, et un des plus utilisés pour le **Deep Learning**.

Sa popularité est liée à :

- sa grande compatibilité (Linux, Mac OS, Android et iOS!)
- l'éventail d'interfaces de programmation possibles (Python, la plus complète, C++, Java et Go)
- ses temps de compilation très courts (grâce à la base en C/C++)
- ses supports de calculs : CPU et GPU
- sa très bonne documentation
- c'est la technologie de Deep Learning de Google

La particularité de TensorFlow est qu'il représente les calculs sous la forme d'un graphe d'exécution : chaque noeud représente une Operation à réaliser, et chaque lien représente un Tensor. Une Operation peut aller d'une simple addition à une fonction complexe de différenciation matricielle. Chaque Operation prend en entrée zéro, un ou plusieurs Tensor, effectue un calcul, et retourne zéro, un ou plusieurs Tensor. Un exemple typique de Tensor est un lot d'images. Un lot d'images est représenté par un Tensor à 4 dimensions : taille du lot (nombre d'images dans le lot), hauteur, largeur et nombre de canaux de représentation (3 pour une image en couleurs représentée en RGB).

La création du graphe est automatiquement gérée par TensorFlow une fois les Tensor et Operation implémentés et instanciés. Cela permet une optimisation et parallélisation du code et de l'exécution lors du lancement.

TensorFlow possède de plus un support très vaste pour la création d'opérations spécifiques au Deep Learning, et il devient donc facile de construire un réseau de neurones et d'utiliser les opérations mathématiques couramment associées pour l'entraîner avec les bons optimiseurs.

Pour effectuer les calculs, un graphe doit être lancé dans une Session. Une Session place les Operations du graphe dans des devices (CPU ou GPU) et met à disposition des méthodes pour les exécuter. Chaque Session peut avoir ses propres variables et readers, et il est possible d'instancier plusieurs Session afin d'entraîner plusieurs réseaux différents. Le lancement des opérations du graphe se fait via la méthode `run()` de la Session. Un graphe ne va exécuter les Operation qu'après la création d'une Session.

Ce système d'exécution de graphe est une des propriétés fondamentales de TensorFlow. Cela permet d'éviter de retourner dans le monde Python à chaque étape (contrairement à ce qui est fait dans NumPy) et d'exécuter toutes les opérations du graphe en une seule fois dans un même environnement optimisé.

Pour des exemples détaillés voir l'article TensorFlow Deep Learning de Yoann BENOIT [5].

2.3.2 OpenCV

Opencv est une bibliothèque graphique open source i.e. qui propose de nombreuses fonctionnalités pour traiter des éléments graphiques. Parmi elles, le traitement d'image (niveaux de gris, lissages, filtrage,...) traitement vidéo (détection et suivi de formes, de visages, de mouvement,...), des algorithmes d'apprentissage (réseaux neuronaux,...) ainsi que du calcul matriciel. Ce sont les fonctionnalités de reconnaissance d'objets qui nous intéressent pour la reconnaissance d'image embarquée. Nous pouvons également entraîner nous-même l'algorithme pour lui faire reconnaître des objets spécifiques mais nous sommes fixés de n'utiliser que des formes simples, déjà pré-apprises. [30]

Cette bibliothèque est écrite en C++ mais peut être aussi utilisée en Python, Java, et MATLAB/OCTAVE.

Pour détecter des objets, OpenCV utilise la méthode de Viola et Jones fonctionnant grâce à un apprentissage supervisé.

Apprentissage : pour cette étape, on utilise une image connue positive ou négative (l'objet à détecter y est présent ou non)

L'image est analysée point point par point.

Pour un point : une zone de détection 24x24 pixels est définie autour de ce point et est agrandie à chaque étape jusqu'à ce que l'image soit entièrement dedans. A chaque itération, ce point est associé à une caractéristique simple : la caractéristique pseudo-Haar calculée par la différence des sommes de pixels de deux ou plusieurs zones rectangulaires

adjacentes. Le calcul des caractéristiques est fortement accéléré par l'utilisation de la méthode de l'"image intégrale". Ensuite, les meilleures caractéristiques sont sélectionnées pour en faire des classifieurs qui sont entraînés par *boosting* : ils sont combinés avec un grand nombre de caractéristiques "faibles" afin de les renforcer. C'est le principe des cascades de Haar.

Les classifieurs sont entraînés à l'aide d'un très grand nombre d'images positives et négatives jusqu'à ce que celui-ci donne des réponses satisfaisantes (non satisfaisant = une réponse aléatoire, satisfaisant = une corrélation importante entre la réponse et la valeur réelle de l'image, en fonction des exigences du cahier des charges).

Pour alimenter cet apprentissage, des bases de données mettent à disposition de grandes quantités d'images. Par exemple ImageNet [17] contient 14 millions d'images étiquetées et accessibles sur demande pour de la recherche.

Détection : elle se fait point par point. A chaque point, une fenêtre 24x24 pixels est initialisée autour du point : tant que le classifieur donne une réponse positive, la fenêtre est agrandie et on réitère. Si tous les étages sont positifs, l'image est positive.

Pour plus de détails, voir la page Wikipedia sur la méthode de Viola et Jones [29].

2.3.3 Keras

Keras est une bibliothèque open source écrite en Python permettant la constitution rapide de réseaux neuronaux. Ainsi, Keras ne fonctionne pas comme un framework propre mais comme une interface de programmation applicative pour l'accès et la programmation de différents frameworks d'apprentissage automatique.

D'après le Ionos Digital Guide [14], Keras est une bibliothèque agissant au niveau du modèle : elle met à disposition des modules permettant de développer des modèles de deep learning (apprentissage profond) complexes. Contrairement aux frameworks indépendants, ce logiciel open source ne s'occupe pas des opérations « low level » et utilise à cet effet les bibliothèques de frameworks d'apprentissage automatique associés qui tiennent pratiquement lieu de moteurs back-end (i.e. sous-jacents) pour Keras. Conformément au principe de modularité, les couches désirées pour le réseau neuronal à mettre en place sont connectées. Pour autant, il n'est pas nécessaire de comprendre l'infrastructure réelle du framework choisi et l'utilisateur de Keras n'a pas à la démarrer directement.

Ces opérations low-levels pourront être exécutées par l'infrastructure de notre choix,

par exemples TensorFlow vu précédemment, ou par une combinaison de celles-ci.

Les autres avantages de Keras pour notre PE sont de proposer :

- une interface modulable, conçue pour l'homme d'abord et de façon secondaire pour les machines afin de simplifier sa prise en main,
- une compatibilité avec de nombreuses plateformes (IOS, Android, Raspberry PI)

2.3.4 YOLO

You Only Look Once est une famille de modèles de reconnaissance d'objets utilisant un **réseau neuronal convolutif**. [26]

Comportement du réseau de neurone Yolo :

Tout d'abord, une grille est appliquée sur l'image afin de générer des *anchor boxes*. Une *anchor box* est un boîte de forme et taille "idéales" positionnée à l'endroit "idéal" pour l'objet qu'elle est chargée de détecter. Elles permettent au réseau de détecter plusieurs objets, à plusieurs échelles, et éventuellement qui se chevauchent.

Ensuite l'algorithme filtre les *anchor boxes* qui ont une faible probabilité de détecter un objet et celles qui détectent le même objet (par la méthode de la Non-maximum Suppression).

Architecture : elle est composée de 24 **couches convolutives** suivies de 2 couches entièrement connectées :

- les 20 premières couches convolutives sont pré-entraînées avec des bases de données comme ImageNet
- les 4 dernières ainsi que les 2 couches entièrement connectées sont ajoutées pour entraîner le réseau à la détection d'objets d'une base de donnée personnalisée.

C'est un entraînement par transfert (*transfer learning*) : un modèle déjà entraîné sur une tâche assez générique est utilisé et ensuite spécialisé sur une tâche d'intérêt en re-entraînant uniquement les dernières couches du réseau avec des données spécifiques à cette tâche.

2.3.5 Comparaisons

OpenCV vs Keras : OpenCV est optimal pour la vision par ordinateur en temps réel, ce qui nous intéresse, tandis que Keras a pour objectif de faciliter la manipulation de réseaux neuronaux. OpenCV existe depuis plus longtemps et bénéficie d'une plus grosse communauté et donc de potentiels tutoriels en ligne. En terme de performances, OpenCV est environ 1,5 fois plus rapide que Keras. La raison est que Keras est codé en python ce qui le rend plus facile à manipuler mais le rend plus lent, tandis qu'OpenCV est codé en C++ qui est un langage de programmation bien plus rapide. Au niveau documentation, Keras est largement supérieur.

OpenCV vs Tensorflow : de même que pour Keras, Tensorflow est plus adapté pour mettre en place des réseaux neuronaux de manière générale tandis qu'OpenCV se concentre sur leurs applications en reconnaissance d'image.

OpenCV vs YOLO : OpenCV et Yolo ont tous deux pour fonctionnalité principale la détection d'objets. YOLO utilise une méthode particulièrement rapide.

Sa méthode de *transfer learning* très intéressante permettrait d'entraîner notre algorithme sur nos propres objets. Toutefois, nous nous sommes fixés de n'utiliser que des objets très simples comme cibles (des formes géométrique simple, de couleurs unies) nous n'aurons donc pas besoin de faire de l'apprentissage.

Enfin, YOLO est intégré à la bibliothèque d'OpenCV et la plupart des tutoriels pour utiliser YOLO nous indique de passer par OpenCV. C'est pour cela que nous allons dans un premier temps nous familiariser avec OpenCV et ce qu'il propose comme fonctionnalités de reconnaissance d'objets. A priori, pour nos besoins assez simples, OpenCV devrait être amplement suffisant mais il est envisageable dans un 2e temps d'utiliser Yolo afin de voir s'il permet d'atteindre de meilleures performances.

En conclusion, nous utiliserons OpenCV pour implémenter de la reconnaissance d'image sur notre drone autonome. Cependant, nous retenons que Yolo est une bonne solution que nous pourrions utiliser si OpenCV seul n'est pas assez performant.

2.3.6 Choix du langage de programmation

Pour choisir le langage de programmation, deux contraintes principales se présentent à nous :

- la rapidité : en effet, nous cherchons à faire de la reconnaissance d'image en temps réel et embarquée sur un drone en mouvement, il faut donc que les opérations s'effectuent rapidement afin d'avoir un système réactif.
- la consommation d'énergie : en effet, la raspberry qui effectuera les calculs sera alimentée par la batterie très limitée du drone. Il faut donc que nos programmes soient le moins énergivores possible afin de garantir l'autonomie de vol minimale souhaitée.

Des chercheurs d'universités au Portugal ont comparé différents langages de programmation sur ces deux critères et sont arrivés aux résultats suivants :

	Energy		Time
(c) C	1.00	(c) C	1.00
(c) Rust	1.03	(c) Rust	1.04
(c) C++	1.34	(c) C++	1.56
(c) Ada	1.70	(c) Ada	1.85
(v) Java	1.98	(v) Java	1.89
(c) Pascal	2.14	(c) Chapel	2.14
(c) Chapel	2.18	(c) Go	2.83
(v) Lisp	2.27	(c) Pascal	3.02
(c) Ocaml	2.40	(c) Ocaml	3.09
(c) Fortran	2.52	(v) C#	3.14
(c) Swift	2.79	(v) Lisp	3.40
(c) Haskell	3.10	(c) Haskell	3.55
(v) C#	3.14	(c) Swift	4.20
(c) Go	3.23	(c) Fortran	4.20
(i) Dart	3.83	(v) F#	6.30
(v) F#	4.13	(i) JavaScript	6.52
(i) JavaScript	4.45	(i) Dart	6.67
(v) Racket	7.91	(v) Racket	11.27
(i) TypeScript	21.50	(i) Hack	26.99
(i) Hack	24.02	(i) PHP	27.64
(i) PHP	29.30	(v) Erlang	36.71
(v) Erlang	42.23	(i) Jruby	43.44
(i) Lua	45.98	(i) TypeScript	46.20
(i) Jruby	46.54	(i) Ruby	59.34
(i) Ruby	69.91	(i) Perl	65.79
(i) Python	75.88	(i) Python	71.90
(i) Perl	79.58	(i) Lua	82.91

FIGURE 6 – Score normalisé de consommation de différents langages [24]

On constate que C++ obtient un très bon score et est intéressant car on peut l'utiliser pour implémenter OpenCV à notre algorithme, tandis que Python par exemple, fait partie des langages les plus lents et les énergivores.

C'est donc C++ qu'il faut idéalement choisir pour notre PE. Cependant, comme nous approchons de la fin de l'année au moment de ce choix, nous n'avons pas le temps d'ap-

prendre le C++ qui est pour nous un nouveau langage. Nous effectuerons donc nos tests avec Python, qui un langage que nous maîtrisons déjà.

2.4 Algorithmes de décision

Le drone doit pouvoir réagir aux informations captées sur l'environnement, c'est-à-dire prendre des décisions et planifier la suite d'actions nécessaire pour remplir sa mission, tout en évitant les obstacles sur son chemin.

Au cours des recherches effectuées, un certain nombres de stratégies de décision pour l'évitement d'obstacle ont été répertoriées [21]. Les différents algorithmes relevés sont présentés dans les sections suivantes.

2.4.1 Algorithme "Bug"

la première classe d algorithmes étudiée est la classe des algorithmes "bug" ; le fonctionnement du plus simple d'entre eux est le suivant :

- le drone se dirige dans la direction de sa cible;
- un obstacle se présente sur la trajectoire du drone;
- l'aéronef effectue un tour complet de l'obstacle;
- une fois que le drone a déterminé le point de l'obstacle le plus proche de son objectif, il recommence à contourner l'obstacle jusqu'à l'atteindre à nouveau;
- une fois atteint ce point, le drone reprend sa route en direction de son objectif initial à atteindre.

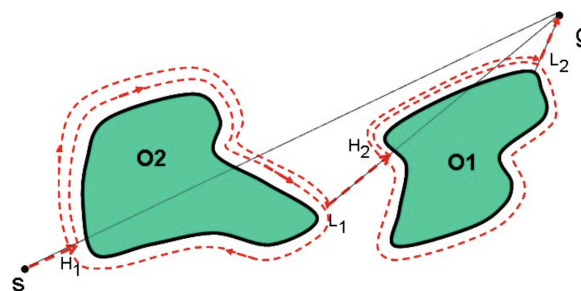


FIGURE 7 – Algorithme "Bug" de base [21]

Plusieurs variantes de cet algorithme existent et sont détaillées un article du IJERT [21]. Une variante plus optimisée de cet algorithme est la suivante :

- initialement, le drone détermine la position et l'orientation de la droite reliant sa position à son objectif ;
- le drone part en suivant cette direction ;
- lorsque le drone se retrouve face à un obstacle, il se met à contourner l'obstacle ;
- dès que le drone retombe sur un point appartenant à la droite initiale, il recommence à la suivre.

Cette méthode est décrite comme étant plus optimisée la plupart du temps, mais pouvant devenir non-optimale dans certains cas.

Une dernière variante de l'algorithme "Bug" est présentée dans "Comparison of Various Obstacle Avoidance Algorithms"[7] :

- le drone part dans la direction de sa cible ;
- lorsqu'il rencontre un obstacle, il se met à le contourner, en calculant à chaque instant la pente de la droite qui le lie à sa cible ;
- dès que cette droite ne rencontre plus l'obstacle, le drone arrête de contourner l'obstacle et se met simplement à suivre la trajectoire donnée par cette droite passant par le drone et sa cible.

2.4.2 Méthodes de machine learning

Des recherches ont déjà été faites pour déterminer l'efficacité des méthodes de *machine learning* pour l'évitement d'obstacles [19].

Néanmoins, trois problèmes se sont posés lors de l'étude des solutions à implémenter :

- l'entraînement automatique implique toujours énormément d'essais ratés ; le drone à disposition n'aurait pas pu encaisser des centaines d'essais à la suite, il serait devenu inutilisable très rapidement ;
- ensuite, l'apprentissage automatique est extrêmement chronophage : le temps d'apprentissage est en effet très long, car il faut souvent effectuer de très grands nombres d'essais avant d'avoir des résultats satisfaisants (du millier aux dizaines de milliers de répétitions) ;
- enfin, il aurait été possible de récupérer les résultats d'expériences déjà menées sur un drone, pour s'épargner le temps d'entraînement de l'algorithme ; cependant, ces résultats sont toujours très spécifiques au système sur lesquels ils ont été réalisés. Ainsi, récupérer un algorithme d'évitement d'obstacle entraîné sur un drone même

légèrement différent au niveau de la masse, la répartition de la masse, ou encore la puissance de chacun des moteurs n'aurait probablement jamais fonctionné sur le drone utilisé dans ce PE (qui est spécifique par le modèle acheté, les composants rajoutés dessus comme ses pieds, le micro-ordinateur et les capteurs).

2.4.3 Méthode du champ de potentiel virtuel

Le dernier type de méthode étudié est celui des champs de potentiel virtuels. La méthode de base fonctionne de la façon suivante :

- on commence par attribuer un potentiel attractif ;
- puis, le drone se met à acquérir en continu les distances le séparant des objets qui l'entourent ;
- à chaque point dont on connaît la distance, on attribue un potentiel répulsif, d'autant plus fort qu'il est proche du drone ;
- en se fondant sur les positions des objets autour de lui et des potentiels à disposition, le micro-ordinateur de vol calcule le vecteur gradient du champ de potentiel dans lequel il se trouve ;
- l'opposé de ce vecteur donne la direction qui minimise le potentiel virtuel du drone, c'est-à-dire la direction qui minimisera la distance entre le drone et sa cible, tout en ne se rapprochant pas trop des obstacles qui l'entourent : le micro-ordinateur de vol donne donc l'instruction au drone de se déplacer dans cette direction.

Les avantages de cette technique sont qu'elle est relativement peu coûteuse en ressources de mémoire et de calcul (par rapport à un traitement d'image classique pour identifier d'éventuels obstacles), et permet de s'assurer que le drone ne rentre pas dans des objets même s'il ne les a pas identifiés clairement. Ces caractéristiques sont celles qui nous ont amené à décider d'implémenter cette méthode plutôt que les algorithmes présentés avant.

2.5 Bilan et choix

L'état de l'art des technologies de drones présente donc un large panel de solutions de stabilisation, de détection et reconnaissance de l'environnement ainsi que d'évitement d'obstacle. Par la suite, l'implémentation et les essais concerne les solutions retenues pour l'étude expérimentale :

- Les capteurs utilisés ici pour la stabilisation sont ceux inclus dans le contrôleur de vol (cf. 2.1.1 i) qui couvrent une grande partie des capteurs présentés en 2.1.1. Notamment, les centrales inertielles apportent beaucoup d'information sur les déplacements du drone dans l'espace afin de mettre en place la commande.
- La commande de la position du drone est effectuée par un correcteur PID (cf 1.1.2 c). Le PID, étant la solution la plus largement utilisée dans l'industrie pour sa simplicité de fonctionnement et ses performances satisfaisantes, est implémenté d'office sur le PX4. Il est réglé par défaut avec des paramètres correspondant à l'architecture du drone utilisé, mais ces paramètres peuvent être choisis par l'utilisateur pour stabiliser le drone. Cette solution, qui ne requiert donc pas de modifier le code source du PX4 pour implémenter une nouvelle méthode de commande, est celle retenue pour la suite de l'étude.
- L'étude de repérage de l'environnement sera effectuée sur une maquette et non sur le drone lui-même afin de ne pas abîmer les capteurs et de ne pas perdre trop de temps sur la réalisation du système de fixation puisque la contrainte de poids imposée par le drone est retirée. Les capteurs retenus sont le télémètre à ultrasons et la caméra de profondeur. Ce qui motive ce choix est le faible coût des capteurs à ultrasons d'une part. En effet, les essais envisagés impliquent l'utilisation de 8 capteurs à ultrasons pour détecter les obstacles dans toutes les directions. D'autre part, la caméra de profondeur a été choisie car elle complète bien les capteurs à ultrasons : elle donne une information sur l'avant uniquement mais de beaucoup plus grande précision et à plus grande distance. Là où les capteurs à ultrasons donnent des informations peu précises et à plus courte distance dans toutes les directions. Pour diminuer les coûts et augmenter la flexibilité dans le développement, les 2 composants de la caméra de profondeur seront commandés séparément, ce qui est moins cher qu'une solution sur étagère.
- La reconnaissance d'image sera effectuée avec la bibliothèque OpenCV utilisée avec le langage Python sur la RaspberryPi. L'image provenant de la caméra Arducam reliée au port CSI de la RaspberryPi. Le langage C++ a également été considéré pour ses meilleures performances, dont son coût énergétique moindre qui est intéressant pour l'autonomie du drone. Cependant, aucun membre du projet ne maîtrisait bien ce langage, et la RaspberryPi est plus facilement utilisable avec le langage Python. Par conséquent, pour faciliter l'implémentation et le gain de temps en fin de projet, le langage Python a été retenu.

2.6 Budget et achat matériel

Afin d'implémenter les choix décrits ci-dessus, commander du matériel a été nécessaire :

- une caméra de profondeur Intel Realsense D420 et son processeur associé pour avoir un flux vidéo de profondeur.
- une dizaine de télémètres à ultrasons dans l'objectif d'avoir des informations sur les obstacles à proximité dans toutes les directions (8 + 2 de marges).
- divers câbles et connectique nécessaires, notamment un câble CSI pour relié la caméra Arducam à la RaspberryPi.

Le détail est donné dans ce tableau :

Pièce	Quantité	Coût unitaire HT	Coût total TTC
Realsense D4 vision processor Board	1	49,90 €	59,88 €
HC-SR04 Ultrasonic Sonar	10	3,59 €	43,08 €
Depth Module de Realsense D420	1	62,79 €	75,35 €
Câble USB 3.1 vers USB-C	1	8,79 €	10,55 €
Wire Jumpers M-F 15 cm x10	2	2,73 €	6,55 €
Wire Jumpers M-M 15 cm x10	2	2,73 €	6,55 €
Wire Jumpers M-M 15 cm x10	5	2,73 €	16,38 €
Câble pour Camera RaspberryPi 15 cm	3	1,77 €	6,37 €
Coût total			224,71 €

TABLE 2 – Budget

Les dépenses sont donc dans les limites du budget du PE (300€).

3 Implémentation et matériel

Une fois les solutions choisies, il faut s'intéresser à la façon de les mettre concrètement en place. Cette partie détaille le matériel utilisé et la façon de l'utiliser pour remplir nos objectifs.

3.1 La stabilisation PID du PX4

En règle générale il est possible de configurer le PID du PX4 (voir l'annexe 6.4 pour plus de détails) de manière automatique. L'autotuning se fait pendant le vol. Cependant, le drone doit voler de manière suffisamment stable pour permettre à l'autotuning de fonctionner correctement. Il faut donc passer par une phase de pré-réglage. Notre référence est donc la documentation du PX4 [3].

3.1.1 Le pré-réglage

Durant cette étape, il faut tout d'abord faire décoller le drone et le faire planer à une distance d'environ un mètre. On utilise ensuite la télécommande pour perturber le vol du drone. Si pour chaque direction, la perturbation est amortie après deux oscillations, on peut passer à l'étape de l'autotuning. Si les oscillations persistent il faut alors reconfigurer le drone.

3.1.2 L'autotuning

Les étapes de l'autotuning [1] sont :

1. Faire le test de pré-réglage
2. Faire décoller le drone et planer entre 4 et 20 mètres d'altitude.
3. Dans le logiciel QGroundControl, ouvrir le menu : Vehicle setup > PID Tuning
4. Sélectionner l'onglet contrôleur de vitesse ou contrôleur d'attitude et s'assurer que le bouton autotune est actif.
5. Cliquer sur le bouton d'autotune et laisser le drone faire des mouvements de manière automatique pour le réglage du PID. Une barre de progression s'affiche et permet de suivre le paramétrage.
6. Une fois l'autotuning terminé, il faut faire atterrir le drone et le désarmer pour appliquer les nouveaux paramètres.

L'autotuning ne suffit pas toujours à assurer la stabilité du drone, auquel cas les paramètres du PID doivent être déterminés manuellement.

3.1.3 Paramétrage manuel du PID

La configuration du réglage PID de QGroundControl [2] fournit des tracés en temps réel des courbes de consigne et de réponse du véhicule. L'objectif du réglage est de définir les valeurs P/I/D de sorte que la courbe de réponse corresponde le plus possible à la courbe de consigne (c'est-à-dire une réponse rapide sans dépassement).

Les correcteurs sont superposés, ce qui signifie qu'un correcteur de niveau supérieur transmet ses résultats à un correcteur de niveau inférieur. Le correcteur de niveau le plus bas est le correcteur des vitesses angulaires (lacet, tangage et roulis), suivi du correcteur d'attitude, et enfin du correcteur de vitesse et de position. Le réglage du PID doit être effectué dans le même ordre, en commençant par le correcteur des vitesses angulaires, car il affectera tous les autres contrôleurs.

La procédure de test pour chaque contrôleur (vitesses angulaires, attitude, vitesse/positions) et axe (lacet, roulis, tangage) est toujours la même : créer un changement de consigne rapide en déplaçant les sticks de la manette très rapidement et observer la réponse. Il faut ajuster ensuite les curseurs (voir figure ci-dessous) pour améliorer le fait que la réponse (courbe bleue ci-dessous) suive la consigne (courbe verte ci-dessous).

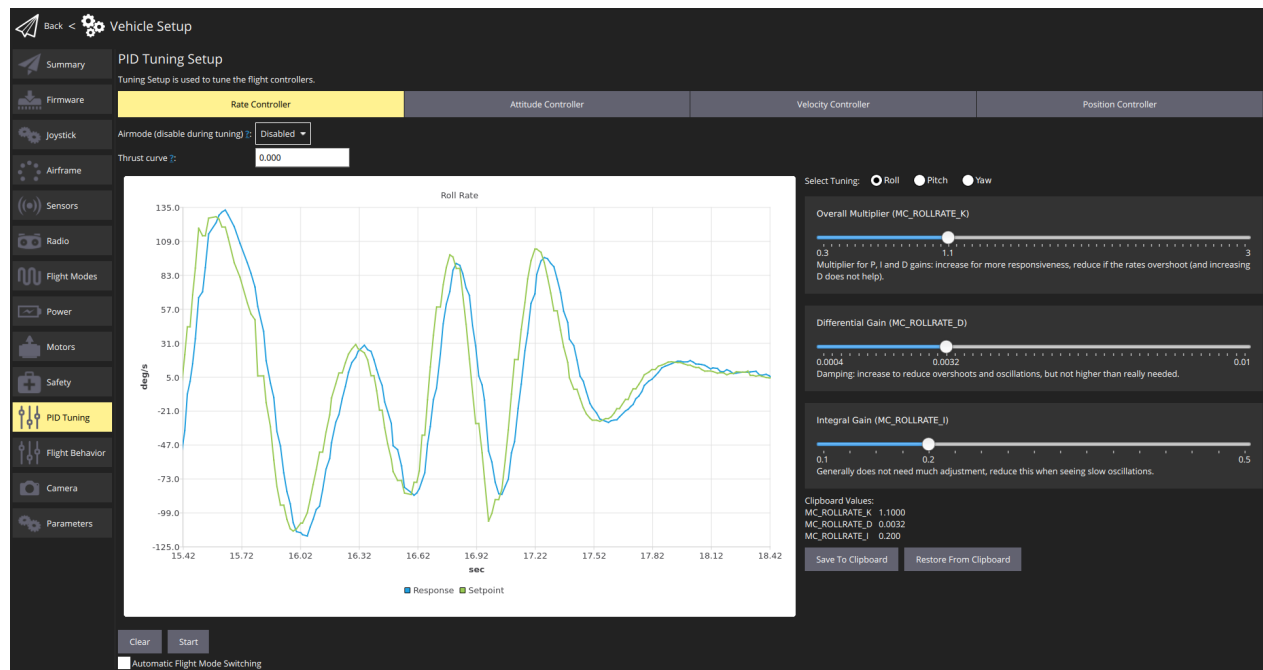


FIGURE 8 – Panel de réglage du contrôleur de vitesse

La procédure complète de configuration est détaillée dans la documentation du PX4.

3.2 Utilisation de la RaspberryPi

Nous disposons d'une RaspberryPi 4 Modèle B 4 Go qui est au centre de l'architecture pour l'évitement d'obstacles.

3.2.1 Architecture pour l'évitement d'obstacles

La RaspberryPi reçoit les informations envoyées par les capteurs, à savoir la caméra de profondeur, la caméra classique et les télémètres à ultrasons. Elle les traite avec un algorithme de prise de décision qui est à préciser, notamment au niveau de la combinaison des différents flux d'information. Des ordres sont ensuite donnés au contrôleur de vol en fonction des résultats de l'algorithme. Le schéma 21 résume l'architecture.

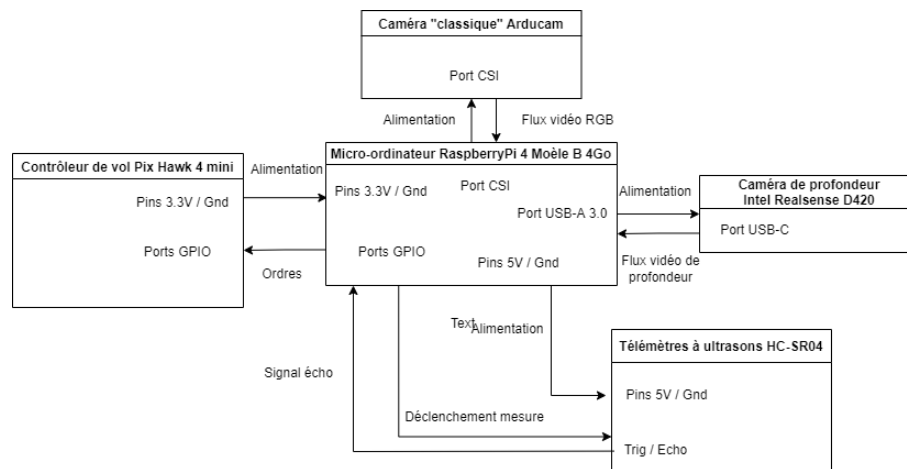


FIGURE 9 – Schéma de l'architecture pour l'évitement d'obstacles

3.2.2 Interfaces Capteurs - RaspberryPi

L'interface entre la RaspberryPi et les capteurs à ultrasons se fait par le biais des ports GPIO (pour "General Purpose In Out") de la RaspberryPi dont la disposition est donnée par le schéma 10.

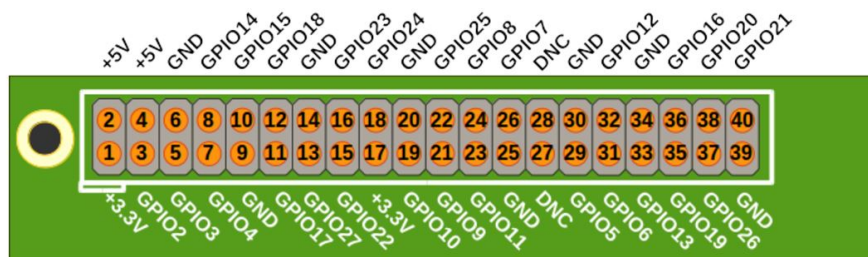


FIGURE 10 – Disposition des pins GPIO de la RaspberryPi 4 B

Les ports de la RaspberryPi peuvent être contrôlés avec des bibliothèques telles que RPi.GPIO sous Python ou WiringPi en C++. Cette dernière est la plus simple d'utilisation en C++. Bien que dépréciée par l'auteur, une branche du projet continue d'être développée et hébergée par la communauté sur Github.

Plus de détails sur l'utilisation de la RaspberryPi, et notamment l'accès à distance qui peut être nécessaire pour plus de flexibilité dans le développement, sont donnés en annexe 6.5.

3.3 Capteurs pour l'évitement d'obstacles

L'évitement d'obstacles nécessite des capteurs afin d'acquérir des informations sur le milieu externe. Cette partie traitera des capteurs choisis et commandés.

3.3.1 Télémètres à ultrasons HC-SR04

Ce capteur est peu cher (moins de 4€ l'unité) et relativement simple à piloter.



FIGURE 11 – Télémètres à ultrasons HC-SR04

Les 4 ports (visibles sur la photos) sont (d'après la documentation technique [27]) :

- Vcc : l'alimentation, DC 5V. Également la valeur du "haut" logique, la valeur basse étant 0V. Courant maximal durant la mesure : 15 mA.
- Trig : le signal de déclenchement pour la mesure, c'est à dire l'émission du chirp ultrasonore. Déclenchement pour une valeur de tension haute pendant plus de $10\mu s$.
- Echo : vaut la tension haute à partir du déclenchement de la mesure. Reste à la tension haute tant que l'écho du signal ultrasonore n'est pas revenu. La durée de ce signal est proportionnelle à la distance entre le capteur et l'obstacle.
- Gnd : la masse du système électronique.

La tension logique haute en sortie, au niveau du port "echo", est de 5V. Pour de nombreux dispositifs comme la RaspberryPi 4 B, la tension admissible sur les ports entrée/sorties (GPIO pour la RaspberryPi) est généralement de 3.3V, donc la tension doit être abaissée avec un pont diviseur de tension.

Les autres caractéristiques sont :

- 40 kHz : fréquence du signal ultrasonore émis.
- 30° : angle du cône de détection.
- 8.7g : masse du détecteur.

— 45 x 20 x 15mm : dimensions hors ports de connexion.

Le but est d'utiliser 8 de ces capteurs disposés sur les côtés d'un hexagone régulier afin de couvrir toutes les directions. Les dimensions de cet hexagone sont déterminées à l'aide d'un modèle géométrique plan réalisé sur le logiciel géogébra 12. Dans ce modèle, les télémètres sont modélisés par des points émettant droit devant eux dans un cône de 30° d'angle (valeur fournie par la documentation technique). Les côtés de l'hexagone ont ensuite été ajusté de sorte à ne laisser aucun angle mort au delà d'un rayon de 25 cm autour du centre de l'hexagone, ce qui correspond approximativement aux dimensions du drone. Un rectangle est laissé libre au milieu afin de mettre en place la RaspberryPi.

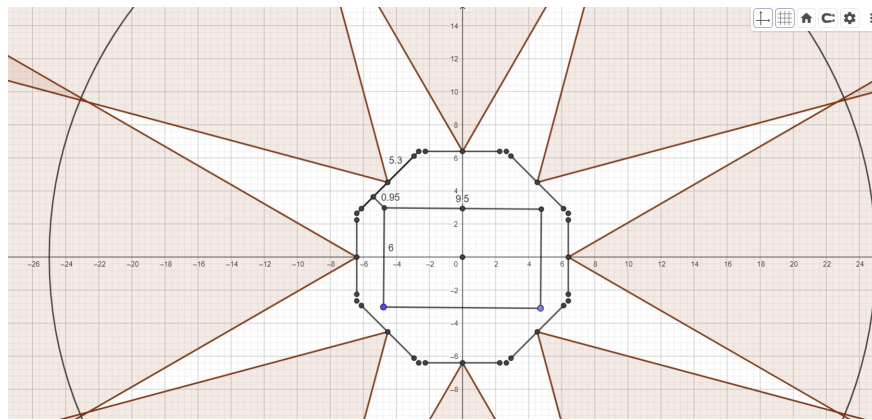


FIGURE 12 – Modèle pour la répartition des télémètres à ultrasons

Le système est ensuite effectivement réalisé 13 par la découpe au laser de plaques de bois de 1cm d'épaisseur :

- 2 hexagones évidés de largeur 0.5cm ;
- 1 hexagone plein.

Ces plaques sont collés et les capteurs sont ensuite fixé à l'aide de systèmes vis-écrous sur chacune des faces de l'hexagone où des trous ont été percés au préalable.

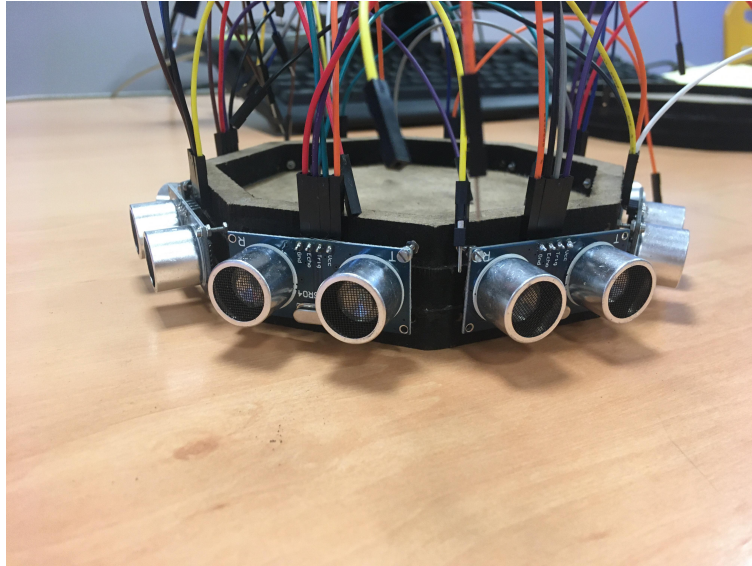


FIGURE 13 – Photographie du système de fixation des télémètres à ultrasons

Le code permettant la mesure de distance pour un capteur seul est situé en annexe 6.7.1.

3.3.2 Intel Realsense D420

Il s'agit d'une caméra de profondeur développée par Intel, de la série D400 conçue pour fonctionner avec son SDK (*Software Development Kit*) disponible sur Github : <https://github.com/IntelRealSense/librealsense>. Elle se décompose en 2 parties : une carte électronique accueillant le processeur de calcul (Vision Processor D4 Board) et un ensemble contenant 2 caméras grands angles solidaires (Stereo Depth Module D420). Le processeur de calcul s'occupe de recomposer les 2 flux vidéos fournis par les caméras en un flux vidéo de profondeur (noir et blanc, pas de couleurs sur ce modèle). Ces 2 ensembles sont reliées par une paire de connecteurs NOVASTACK 35-P 50 pins (schéma 14). Le processeur de calcul est relié à un ordinateur via un câble USB-A 3.0 (côté ordinateur) vers USB-C (côté processeur de calcul).

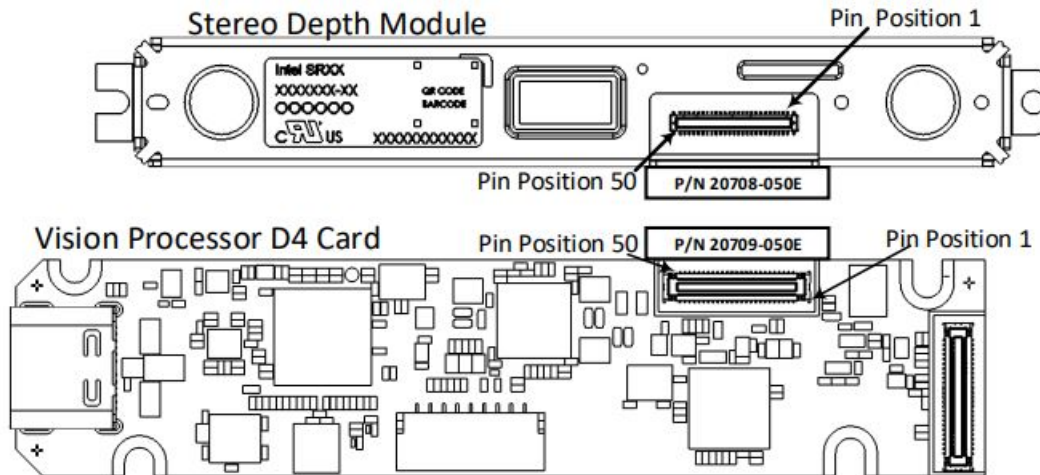


FIGURE 14 – Composants de la caméra de profondeur [16]

Le processeur visuel peut être alimenté et communiquer avec un ordinateur (dans notre cas la RaspberryPi) via son port USB-C (femelle). Un câble USB-C (mâle) - USB-A 3.1 (mâle) est donc nécessaire.

D'après le schéma 14, l'appariement direct des 2 connecteurs NOVASTACK donne lieu à une obstruction du champ de vision des caméras. Un "interposer" existe mais n'a pas pu être commandé avant la fin du projet. La caméra de profondeur ne pourra donc pas être implémentée. Il est disponible chez le fournisseur Mouser : <https://www.mouser.fr/ProductDetail/Intel/82635DSITR50P?qs=wd5RIQLrsJhIJqpBcxiaKQ>.

Les informations présentées ci-dessus sont issus de la documentation technique de la famille de produit D400 de Intel [16].

3.4 Algorithme de reconnaissance d'image

3.4.1 Matériel utilisé

Dans le but d'implémenter la reconnaissance d'image embarquée, nous avons utilisé la raspberryPi à laquelle nous avons branché une caméra (Arducam pour Raspberry PI) sur le port CSI (camera serial interface). Un câble micro HDMI vers HDMI est également utilisé pour visualiser le bureau de la raspberry sur un écran externe.



FIGURE 15 – RaspberryPi équipée d'une caméra sur son port CSI

Le code est rédigé et testé sur un PC indépendant doté d'un interpréteur python, d'OpenCV et d'une webcam avant d'être envoyé sur la raspberry qui l'exécute. Afin de voir ce qu'il se passe lors de l'exécution du code sur la Raspberry, dans une optique de tests, il faut brancher celle-ci à un écran, un clavier et une souris.

3.4.2 Algorithme et code python

Le but du programme est de détecter la cible d'intérêt du drone pour lui permettre de la suivre :

La première étape consistait à installer opencv et python sur la raspberryPi (à l'aide ce tuto [20]). Une fois installés, nous avons développé un algorithme python utilisant la bibliothèque opencv permettant de reconnaître une cible. Le code est disponible en annexe 6.6.1.

Dans un premier temps, il nous faut enclencher la prise de flux vidéo de la caméra tout en spécifiant les propriétés de taille de la fenêtre (qui nous permettra de visualiser les résultats) ainsi que la luminosité de la vidéo. (Lignes 3 à 10)

A chaque frame de la vidéo capturée par la caméra, on récupère l'image en question qui sera traitée par la fonction `findColor()`. (A partir de ligne 34)

La fonction `findColor()` vérifie dans l'image qu'il y'a la présence de la couleur spécifiée dans la variable `couleursDetect` (rouge ici, il suffit d'ajouter une autre liste afin que le programme détecte une autre couleur ; une explication sur le paramétrage d'une couleur est donné plus bas). Une fois que la couleur est détectée, la fonction `findColor()` fait appel à la fonction `getContours()`.

La fonction `getContours()` quant à elle va repérer tous les contours associés aux extrémités des zones de couleur détectées. Une fois ces contours détectés, pour chacune d'elles on

vérifie si la surface entourée par ce contour est assez grande et si la forme correspond plus ou moins à une certaine forme (ici un carré). Si ces conditions sont respectées alors la cible est repérée.

Pour paramétrer une couleur on utilise le script `param_couleur` disponible annexe 6.6.2. Ce code consiste à la création d'un "masque" permettant de filtrer les couleurs afin de sélectionner celles qui nous intéressent. Pour configurer ce masque, nous jouons sur certains paramètres (Hue, Sat, Val) à l'aide de "trackbars" et observons le résultat en temps réel sur la filtration de couleurs jusqu'à ce que la bonne couleur soit sélectionnée. Nous notons ensuite les paramètres correspondants, que nous inscrivons dans le programme présenté précédemment (dans le même ordre).

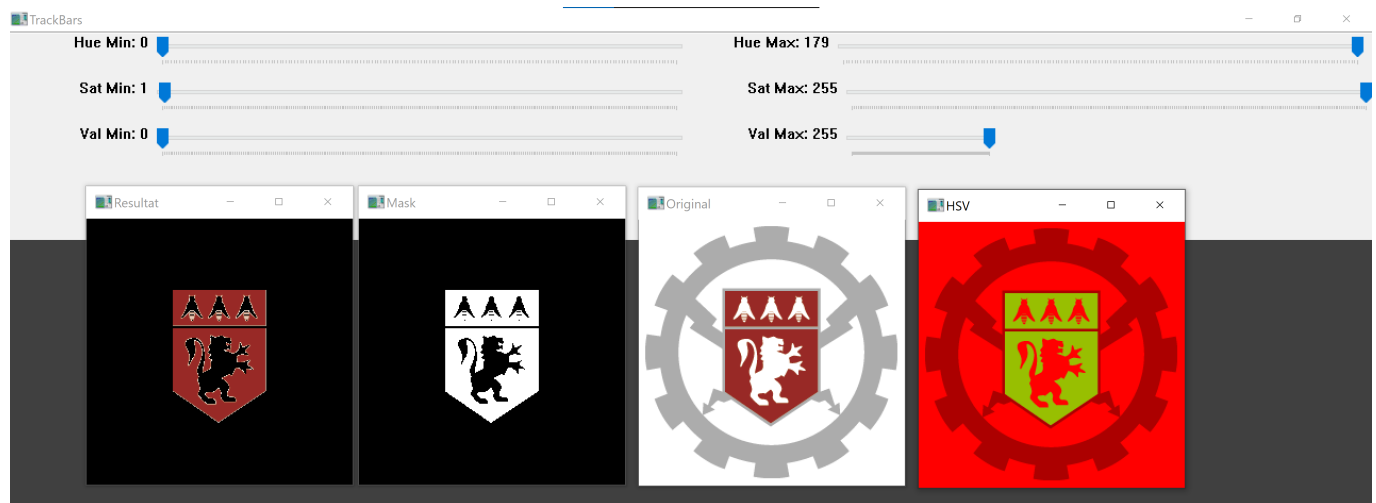


FIGURE 16 – Exemple de paramétrage de la couleur rouge bordeaux

Ces codes ont été réalisés à l'aide d'un tutoriel Youtube OpenCV sur Python [23]. Il existe aussi des tutoriels OpenCV sur C++ [22].

L'étape suivante consiste à positionner le centre de la cible détectée afin de minimiser sa distance au centre de la caméra. En effet, on souhaite mettre en place un asservissement du drone visant à placer le drone en face de la cible (donc au centre de la caméra puisque celle-ci est fixée à l'avant du drone). Pour cela, dans la deuxième partie de la fonction `getContour()`, une fois qu'on a détecté la cible, on calcule les coordonnées du centroïde. On vérifie ensuite que le centroïde est placé dans une certaine zone centrale (ici un carré de 50 pixels de côté) de l'image fournie par la caméra. Si la condition est vérifiée alors on considère que le drone est en face de la cible et notre fonction renvoie la valeur 0. Dans le cas contraire, notre fonction renvoie les valeurs 1, 2, 3 ou 4 selon que la cible se trouve

respectivement à droite, à gauche, en haut ou en bas, ce qui permet au drone de s'orienter dans la bonne direction. Le code correspond à l'annexe 6.6.1.

Maintenant que nous sommes capables de détecter la cible et de placer son centre par rapport à la caméra, nous devons adapter les informations renvoyés par le programme afin de les implémenter dans un programme plus général, prenant en compte l'évitement d'obstacle.

De plus, il faut que des consignes finales telles que "tourner à droite" ou "monter" soient traduites dans un protocole permettant de communiquer avec le contrôleur de vol du drone. Pour cela une piste est donnée dans l'annexe 6.5.2

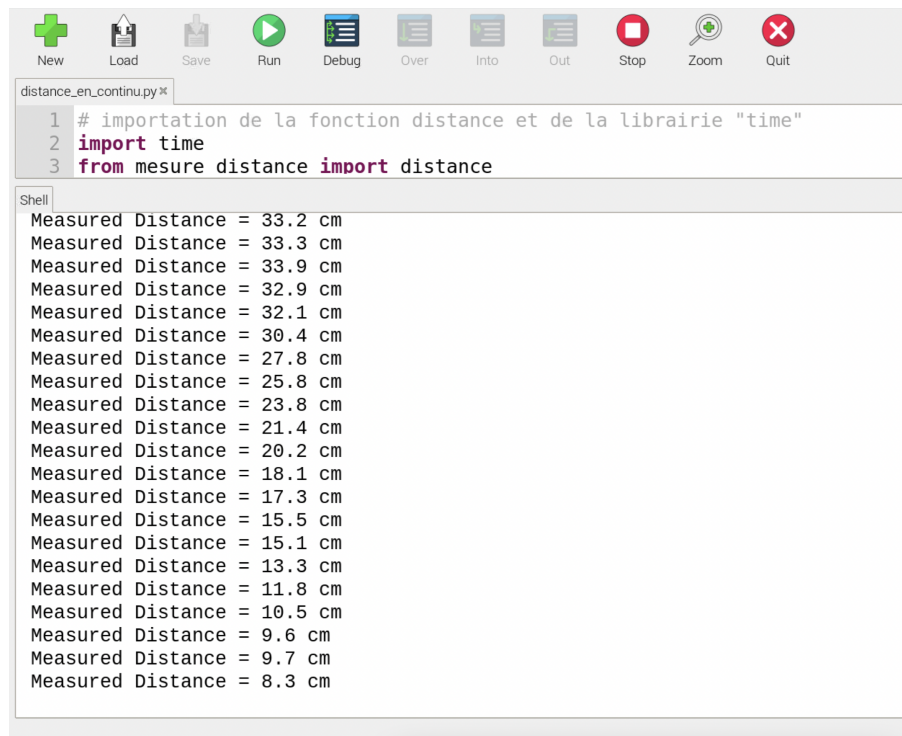
4 Expériences et résultats

4.1 Télémètres à ultrasons

Les expériences suivantes ont porté sur les télémètres à ultrasons, et permettent de mettre en évidence certaines caractéristiques des capteurs.

4.1.1 Premières mesures

La première expérience réalisée était simplement de réaliser des mesures de distances, d'abord une par une, puis en continu. Une fois le code écrit pour réaliser des mesures simples de distance, il a été possible de réaliser des mesures en continu (code en annexe 6.7.2), illustrées dans la capture d'écran ci-dessous¹⁷.



```
distance_en_continu.py x
1 # importation de la fonction distance et de la librairie "time"
2 import time
3 from mesure distance import distance

Shell
Measured Distance = 33.2 cm
Measured Distance = 33.3 cm
Measured Distance = 33.9 cm
Measured Distance = 32.9 cm
Measured Distance = 32.1 cm
Measured Distance = 30.4 cm
Measured Distance = 27.8 cm
Measured Distance = 25.8 cm
Measured Distance = 23.8 cm
Measured Distance = 21.4 cm
Measured Distance = 20.2 cm
Measured Distance = 18.1 cm
Measured Distance = 17.3 cm
Measured Distance = 15.5 cm
Measured Distance = 15.1 cm
Measured Distance = 13.3 cm
Measured Distance = 11.8 cm
Measured Distance = 10.5 cm
Measured Distance = 9.6 cm
Measured Distance = 9.7 cm
Measured Distance = 8.3 cm
```

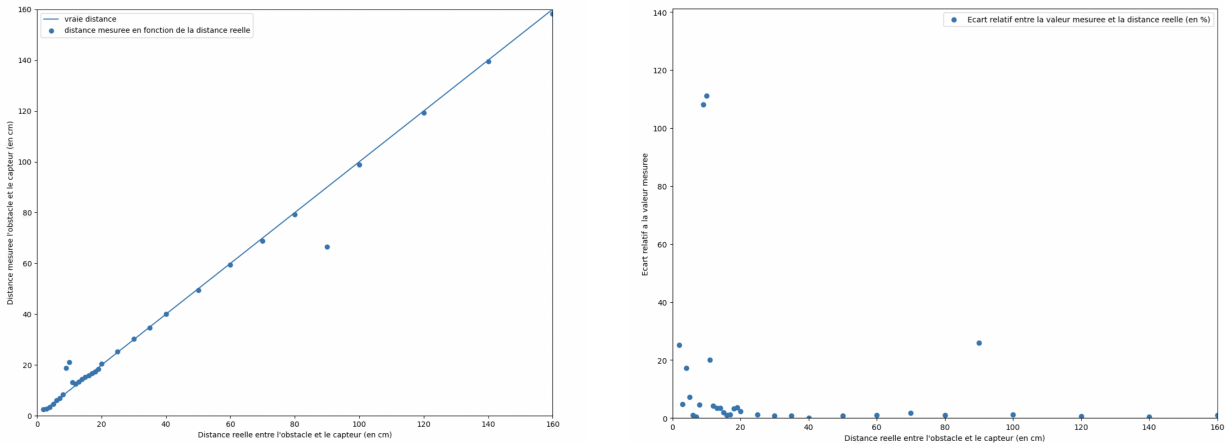
FIGURE 17 – Mesure de distance en continu

Le code dans cette image a en paramètre un intervalle de temps de une seconde entre chaque mesure de distance, et a été lancé alors que l'on rapprochait un obstacle du capteur : l'évolution de ces mesures successives est cohérente avec la situation dans laquelle elles ont été effectuées.

Il est important de noter que pour un obstacle situé à quelques dizaines de centimètres, les mesures en continu ne sont plus fiables lorsque l'intervalle de temps entre chaque mesure passe en-dessous des 0,1 secondes : à partir de cette valeur, les échos des différentes salves d'ultrasons se parasitent entre eux et ne permettent plus d'obtenir des mesures correctes.

4.1.2 Mesures de distances variables

L'expérience suivante a consisté à évaluer la précision des mesures effectuées par un capteur au fur et à mesure que la distance entre l'obstacle et le capteur évolue (le code correspondant se situe en annexe). Un obstacle a été placé en face d'un capteur, puis éloigné progressivement, en relevant à chaque déplacement la distance réelle entre les deux et la distance mesurée par le capteur. Les résultats obtenus sont présentés dans les graphes ci-après¹⁸ :



(a) Distance mesurée par le capteur en fonction de la distance réelle entre le capteur et l'obstacle

(b) Écart relatif entre la distance réelle entre le capteur et l'obstacle et celle mesurée par le capteur

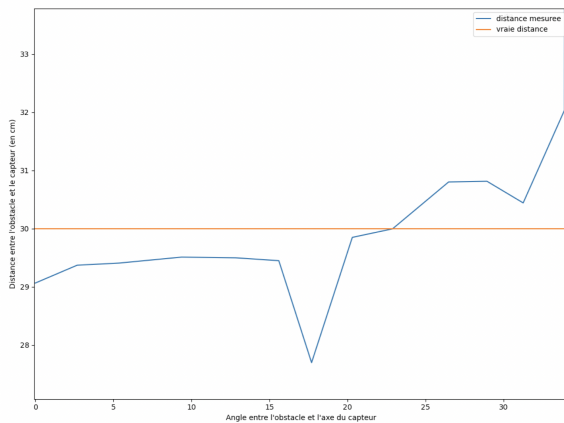
FIGURE 18 – Résultats des mesures de distance effectuées avec le capteur à ultrasons

Globalement, la distance mesurée par le capteur à ultrasons est très proche de celle réelle : les écarts absolus entre la distance réelle et celle mesurée sont de l'ordre du centimètre, ce qui explique que l'écart relatif devienne significativement faible quand l'objet détecté est situé à quelques dizaines de centimètres du capteur. De plus, les distances mesurées ont tendance à être surestimées par rapport aux distances réelles, ce qui est bien moins problématique que le contraire dans le cadre d'une implémentation future sur le drone.

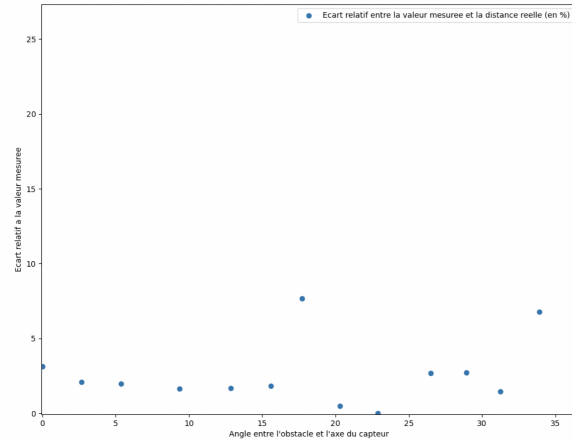
Plusieurs valeurs aberrantes apparaissent, principalement lorsque l'obstacle est proche du capteur : dans la mesure où ces erreurs semblaient apparaître de façon systématique et aléatoire sur chaque série de mesure, il se peut que le protocole de mesure n'ait pas été suffisamment attentif aux perturbations environnantes.

4.1.3 Détermination de l'angle du cône de détection

Cette dernière expérience visait à vérifier la valeur donnée par le fournisseur du capteur quant à sa fiabilité : il était indiqué que le demi-angle du cône de détection du capteur était de 30 degrés. Pour vérifier cela, un obstacle de 7,5cm de largeur a été placé à 30 cm du capteur, à des angles différents par rapport à l'axe du capteur (le code correspondant se situe en annexe 7.7.4). Les résultats obtenus sont présentés dans les graphes ci-après 18 :



(a) Distance mesurée par le capteur en fonction de l'angle entre l'axe du capteur et la droite capteur-obstacle



(b) Écart relatif entre la distance mesurée par le capteur et la distance réelle en fonction de l'angle entre l'axe du capteur et la droite capteur-obstacle

FIGURE 19 – Résultats des mesures de distance en fonction de l'angle entre l'axe du capteur et la droite capteur-obstacle

Conformément aux indications commerciales données par le fournisseur, le capteur est capable de détecter un obstacle jusqu'à 30 degrés par rapport à son axe principal. On observe par ailleurs que l'écart relatif entre la distance réelle à l'obstacle et celle mesurée n'excède quasiment jamais les 5%.

4.2 PID

On réalise les essais en effectuant un vol du drone et en observant ses déplacements en fonction de la consigne (donnée par la manette). Les paramètres du PID doivent être ajustés afin de stabiliser le drone, c'est-à-dire pour que sa position mesurée suive au mieux la consigne. On observe ainsi la rapidité de la réponse du drone, ses éventuels dépassements de consigne et son erreur statique. Cependant, durant les essais réalisés, les courbes de consigne et de réponse n'ont pas pu être acquises correctement sur le logiciel QGroundControl ; les ajustements des paramètres se sont donc faits à partir des observations visuelles des réactions du drone aux commandes.

Au premier vol, les paramètres sont ceux attribués par défaut par le contrôleur de vol à l'architecture du drone quadrirotor. Le premier problème observé est que le drone ne garde pas un vol stationnaire dès le décollage et présente des oscillations de très grande amplitude. Ensuite, le drone oscille très rapidement en roulage en vol stationnaire. Augmenter le gain dérivé en roulage permet alors diminuer les oscillations.

Après les ajustements successifs apportés au PID, le drone est visiblement plus stable qu'au départ. On note en particulier la disparition des oscillations en roulage. Cependant, il reste des erreurs statiques sur la vitesse du drone : le véhicule a tendance à dériver de son point de vol stationnaire, ce qui nécessite l'intervention de l'opérateur pour ramener le drone à sa position (altitude ou position dans le plan horizontal). Si les oscillations en vol sont peu importantes par rapport à la taille caractéristique du drone, le problème de dérivation ne permet pas d'atteindre un vol stationnaire considéré stable.

4.3 Reconnaissance d'image avec OpenCV

4.3.1 Reconnaissance de la cible

Afin de tester l'algorithme de reconnaissance d'image, nous avons branché à un écran la raspberryPi munie de sa caméra via un câble hdmi. Ensuite nous avons paramétré le code python de façon à reconnaître un carré de couleur rouge. Pour cela il suffisait de modifier la variable couleurDetect et d'imposer le nombre de coins du contour à 4. Une fois lancé, le code python a donné le résultat visible dans la figure ci-dessous.

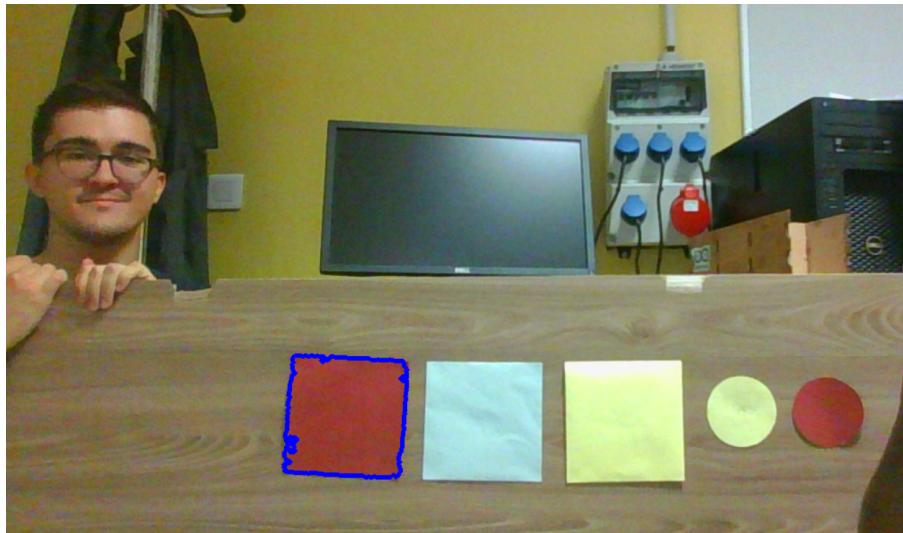


FIGURE 20 – Résultat de l'expérience de la reconnaissance de couleur et de forme (seul le carré rouge est détecté)

Lors des expériences, nous nous sommes rendus compte que cette méthode reste très sensible aux conditions d'éclairage de la pièce. Cela est dû au paramétrage de la couleur présenté précédemment qui dépend fortement de la luminosité et une couleur configurée est moins bien détectée lorsque les conditions d'éclairage changent.

4.3.2 Positionnement de la cible

Pour cette expérience on reste dans les mêmes conditions que pour la reconnaissance de la cible.

- Lorsque le centroïde de la cible est positionné dans le cercle vert centrale, on considère que la cible est bien alignée.
- Lorsqu'il est à gauche ou à droite du cercle centrale, on indique que le drone doit tourner respectivement à gauche ou à droite.
- Lorsqu'il est en haut ou en bas du cercle centrale, on indique que le drone doit respectivement monter ou descendre.

Les résultats sont observables dans la figure ci-dessous. Dans le coin supérieur gauche de chaque image est affichée la consigne adéquate.



FIGURE 21 – Résultat de l'expérience de positionnement de la cible

5 Conclusion

L'objectif du PE était l'étude des moyens de stabilisation, d'évitement d'obstacles et de reconnaissance d'image embarquée à bord d'un drone, en se basant sur le travail du PE précédent. Il s'agissait moins de réaliser un drone fonctionnel que de faire avancer les connaissances sur ces fonctions des drones. En ce sens, l'objectif est accompli. En effet, avec ce rapport, un tableau d'ensemble des solutions a été réalisé et certaines solutions ont pu être testées, notamment :

- un algorithme de reconnaissance d'image permettant au drone de savoir comment corriger son orientation afin de se placer en face d'une cible ;
- la détermination des caractéristiques réelles des télémètres à ultrasons ;

Cependant, ce travail a ses limites. En effet, pour la stabilisation du drone, il est possible d'aller plus loin en développant un module pour le contrôleur de vol permettant de mettre en oeuvre une correction autre que PID, ou d'implémenter le capteur PIX-FLOW qui permet d'avoir une information supplémentaire sur la position du drone. De plus, la caméra de profondeur doit encore être testée et les tests des télémètres à ultrasons peuvent encore être approfondis.

Une autre difficulté qu'il reste à surmonter est l'intégration des différents flux d'informations des différents capteurs dans une stratégie de navigation cohérente. Cela passera probablement par l'implémentation d'un algorithme de décision se basant sur la méthode du gradient de potentiel virtuel et communiquant avec le contrôleur de vol via le protocole Mavlink.

Le but ultime d'un projet dans la continuité de celui-ci serait l'intégration des solutions testées pour créer un drone capable de réaliser un parcours test vers une cible en évitant les obstacles sur son chemin.

Références

- [1] Px4 user guide : Basic configuration : Auto-tuning. Disponible à <https://docs.px4.io/master/en/config/autotune.html> (22/03/2022).
- [2] Px4 user guide : Basic configuration : Mc pid tuning. Disponible à https://docs.px4.io/v1.12/en/config_mc/pid_tuning_guide_multicopter_basic.html (22/03/2022).
- [3] Px4 user guide : Hardware : Pixhawk 4 mini. Disponible à https://docs.px4.io/v1.12/en/flight_controller/pixhawk4_mini.html (24/03/2022).
- [4] Anas ALHASHIMI, Damiano VARAGNOLO et Thomas GUSTAFSSON : Statistical modeling and calibration of triangulation lidars. *ICINCO*, 2016.
- [5] Yoann BENOIT : Tensorflow deep learning – Épisode 1 – introduction. Disponible à <https://blog.engineering.publicissapient.fr/2017/03/01/tensorflow-deep-learning-episode-1-introduction/> (XX/03/2022).
- [6] Guy LE BESNERAIS, Pauline TROUVÉ-PELOUX, Frédéric CHAMPAGNAT et Aurélien PLYER : Capteurs et traitement pour l'imagerie 3d passive. *Technique de l'ingénieur*, 2017.
- [7] Vayeda Anshav BHAVESH : Comparison of various obstacle avoidance algorithms. *International Journal of Engineering Research Technology (IJERT)*, 2015.
- [8] Philippe BONNIFAIT et Clément ZINOUNE : Introduction aux techniques de navigation autonome pour les véhicules intelligents. *Techniques de l'ingénieur*, 2021.
- [9] Gino BRUNNER, Bence SZEVEDY, Simon TANNER et Roger WATTENHOFER : The urban last mile problem : Autonomous drone delivery to your balcony. *IEEE*, 2019.
- [10] Angelo Nikko CATAPANG et Manuel Ramos JR. : Obstacle detection using a 2d lidar system for an autonomous vehicle. *IEEE*, 2016.
- [11] Adrien DROUOT : *Stratégies de commande pour la navigation autonome d'un drone projectile miniature*. Thèse de doctorat, 2013.
- [12] Allen FERRICK, Jesse FISH, Edward VENATOR et Gregory S. LEE : Uav obstacle avoidance using image processing techniques. *IEEE*, 2012.
- [13] Meng GUANGLEI et Pan HAIBING : The application of ultrasonic sensor in the obstacle avoidance of quad-rotor uav. *IEEE*, 2016.
- [14] Ionos Digital GUIDE : Keras : une bibliothèque open source pour la constitution de réseaux neuronaux. Disponible à <https://www.ionos.fr/digitalguide/web-marketing/search-engine-marketing/quest-ce-que-keras/> (XX/03/2022).

- [15] Jia HU, Yifeng NIU et Zhichao WANG : Obstacle avoidance methods for rotor uavs using realsense camera. *IEEE*, 2017.
- [16] INTEL : Realsense d400 series product family. Rapport technique, 2019.
- [17] Stanford Vision LAB, Stanford UNIVERSITY et Princeton UNIVERSITY : Imagenet. Disponible à <https://www.image-net.org/index.php> (XX/03/2022).
- [18] Kenjiro NIWA, Keigo WATANABE et Isaku NAGAI : A detection method using ultrasonic sensors for avoiding a wall collision of quadrotors. *IEEE*, 2019.
- [19] Víctor Mondéjar-Guerra Tiago M. Fernández-Caramés PAULA FRAGA-LAMAS, Lucía Ramos : A review on iot deep learning uav systems for autonomous obstacle detection and collision avoidance. *Multidisciplinary Digital Publishing Institute (MDPI)*, 2019.
- [20] Q-ENGINEERING : Install opencv 4.5 on raspberry pi 4. Disponible ici <https://qengineering.eu/install-opencv-4.5-on-raspberry-pi-4.html> (31/12/2021).
- [21] Maria Isabel RIBEIRO : Obstacle avoidance. 2005.
- [22] Murtaza's Workshop ROBOTICS et AI : Learn opencv c++ in 4 hours. Disponible ici <https://youtu.be/2FYm3G0onhk>.
- [23] Murtaza's Workshop ROBOTICS et AI : Learn opencv in 3 hours with python. Disponible ici <https://youtu.be/WQeo07MI0Bs>.
- [24] Francisco Ribeiro Rui Rua Jácome Cunha João Paulo Fernandes João Saraiva RUI PEREIRA, Marco Couto : Energy efficiency across programming languages. *Universidade do Minho, Univ. Nova de Lisboa, Universidade de Coimbra, Portugal*, 2017.
- [25] Jing SONG, Xu SHUN et Shuang WU : Design of 77ghz narrow beamwidth antenna for uavs obstacles avoidance radar. *IEEE*, 2018.
- [26] Octo TALKS : You only look once – un réseau de neurones pour la détection d'objets. Disponible à <https://blog.octo.com/you-only-look-once-un-reseau-de-neurones-pour-la-detection-dobjets/> (XX/03/2022).
- [27] Cytron TECHNOLOGIES : Hc-sr04 user's manual. Rapport technique, 2013.
- [28] Philip GASSMANN Adrien MANRY Wassim NEJJAR Ferdinand WILLEMIN THÉO LEROY, Tom RENAUDIN : Droneload 2020.
- [29] WIKIPEDIA : Méthode de viola et jones. Disponible à https://fr.wikipedia.org/wiki/M%C3%A9thode_de_Viola_et_Jones (XX/03/2022).

-
- [30] WIKIPEDIA : Opencv. Disponible à <https://fr.wikipedia.org/wiki/OpenCV> (XX/03/2022).
 - [31] Chenming XIN, Guoyou WU, Congyu ZHANG, Kai CHEN, Jinjian WANG et Xiang WANG : Research on indoor navigation system of uav. *IEEE*, 2020.
 - [32] Yuefan XU, Minling ZHU, Yue XU et Mingjie LIU : Design and implementation of uav obstacle avoidance system. *IEEE*, 2019.
 - [33] Yang YU, Wang TINGTING, Chen LONG et Zhang WEIWEI : Stereo vision based obstacle avoidance strategy for quadcopter dav. *IEEE*, 2018.

6 Annexes

6.1 Diagramme de GANTT

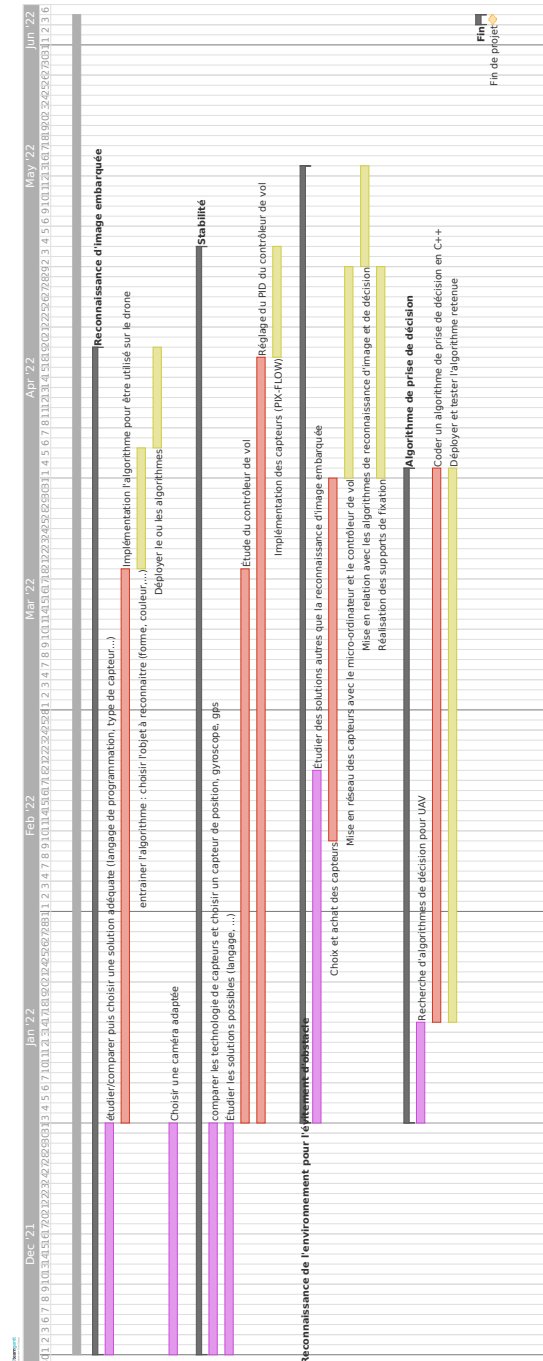


FIGURE 22 – Diagramme GANTT (20/03/2022)

6.2 Tableau des tâches

Tâches terminées	Tâches non-réalisées
• Étudier puis choisir une solution adéquate de reconnaissance d'image • Choisir une caméra adaptée pour la reconnaissance d'image embarquée • Comparer les technologie de capteurs dans une optique de stabilisation et en choisir un ou plusieurs • Étudier les solutions possibles de stabilisation • Étudier le contrôleur de vol • Pour l'évitement d'obstacles, étudier des solutions autres que la reconnaissance d'image embarquée • Choisir et acheter de capteurs pour l'évitement d'obstacle (autres que caméras) • Rechercher des algorithmes de prise de décision pour le drone (dans une optique d'évitement d'obstacles) • Régler le PID du contrôleur de vol • Suivre la livraison du matériel commandé • Implémenter l'algorithme de reconnaissance d'image • Réaliser les supports de fixation des différents capteurs de mesure de distance • Mettre en réseau des capteurs avec le micro-ordinateur dans le cadre de l'évitement d'obstacles • Entraîner l'algorithme de reconnaissance d'image (choisir l'objet à reconnaître) • Choisir un algorithme de prise de décision • Déployer l'algorithme de reconnaissance d'image sur le micro-ordinateur	• Implémenter l'algorithme de prise de décision en Python • Implémenter le capteur PIX-FLOW pour la stabilisation du drone par odométrie visuelle • Mettre en réseau du contrôleur de vol avec le micro-ordinateur • Tester l'ensemble des solutions et quantifier leur performances (partiellement réalisé)

TABLE 3 – Avancement des tâches

6.3 Cahier des charges fonctionnel de l'étude

Fonction	Critère d'appréciation	Niveau de performance à atteindre
Répertoire des solutions existantes de stabilisation de drones, les résumer, et en particulier synthétiser leurs caractéristiques théoriques et opérationnelles.	Nombre de solutions	4 solutions minimum
	Nombre d'articles	2 articles scientifiques / éléments de documentation pertinents par solution, minimum
	Nombre de caractéristiques pour chaque solution	5 minimum
Répertoire des solutions existantes de reconnaissance d'image embarquée, les résumer, et en particulier synthétiser leurs caractéristiques théoriques et opérationnelles.	Nombre de solutions	4 solutions minimum
	Nombre d'articles	2 articles scientifiques / éléments de documentation pertinents par solution, minimum
	Nombre de caractéristiques pour chaque solution	5 minimum
Répertoire des solutions existantes d'évitement d'obstacles, les résumer, et en particulier synthétiser leurs caractéristiques théoriques et opérationnelles.	Nombre de solutions	4 solutions minimum
	Nombre d'articles	2 articles scientifiques / éléments de documentation pertinents par solution, minimum
	Nombre de caractéristiques pour chaque solution	5 minimum
Étudier et comparer les performances des solutions de stabilisation de drones entre elles, et les comparer à la littérature répertoriée.	Nombre de solutions étudiées par expérimentations à l'aide du drone préexistant / à l'aide de simulations	1 minimum
	Nombre de types de tests effectués par solution étudiée	8 par solution minimum
Étudier et comparer les performances des solutions de reconnaissance d'image embarquée entre elles, et les comparer à la littérature répertoriée.	Nombre de solutions étudiées par expérimentations à l'aide du drone préexistant / à l'aide de simulations	1 minimum
	Nombre de types de tests effectués par solution étudiée	8 par solution minimum
Étudier et comparer les performances des solutions d'évitement d'obstacles entre elles, et les comparer à la littérature répertoriée.	Nombre de solutions étudiées par expérimentations à l'aide du drone préexistant / à l'aide de simulations	1 minimum
	Nombre de types de tests effectués par solution étudiée	8 par solution minimum

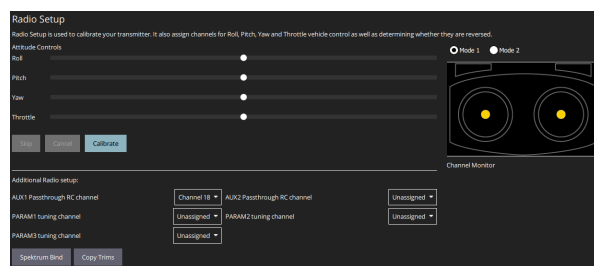
FIGURE 23 – Cahier des charges fonctionnel de l'étude

6.4 Connectique du drone : QGroundControl

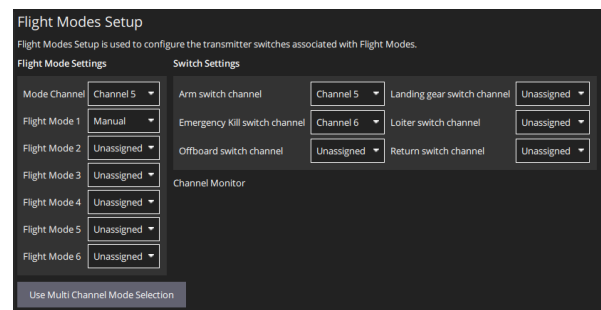
Le contrôleur de vol peut être configuré à l'aide du logiciel QGroundControl qui permet de régler différents types de paramètres. Dans notre cas, comme nous travaillons avec un drone à quatre moteurs, nous devons sélectionner le type : *Quadrotor X*.

Après avoir sélectionné le type de airframe, nous devons effectuer un calibrage de tous les capteurs. Cela peut être fait facilement avec l'aide du logiciel qui donne des instructions à l'utilisateur dans chaque situation.

En ce qui concerne la manette, il y a deux ajustements nécessaires. La partie Radio Setup est destinée à calibrer les joysticks de la manette, et la deuxième partie, Flight Modes, est destinée à attribuer des commandes à chaque bouton de celle-ci.



(a) Radio Setup



(b) Flight Modes

FIGURE 24 – QGround

Le résumé de tous les paramètres choisis peut être visualisé dans la figure 25.

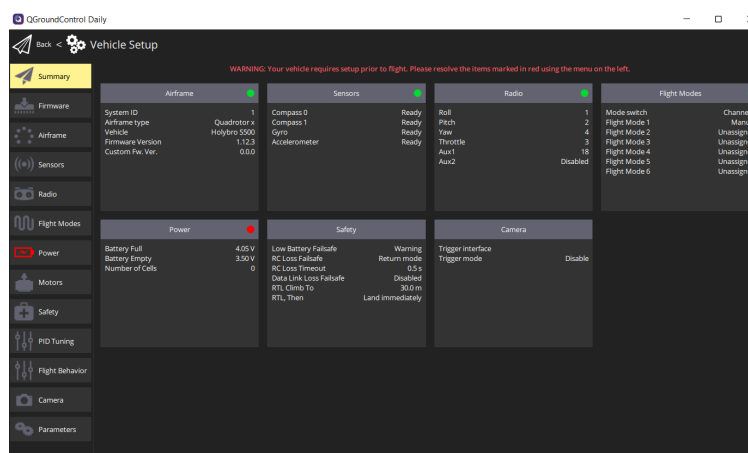


FIGURE 25 – Interface de QGroundControl

6.5 Aide à l'utilisation de la RaspberryPi

6.5.1 Connexion à distance

Il est essentiel de pouvoir accéder à la RaspberryPi, pour pouvoir la configurer et lancer les algorithmes écrits. Un écran, un clavier et une souris peuvent être branchées respectivement sur les port micro-HDMI display, et USB de la RaspberryPi. Néanmoins, pour initialiser les algorithmes lorsque la RaspberryPi sera branchée et attachée au drone, une solution plus pratique est nécessaire. Il est possible d'accéder à distance à la RaspberryPi via le protocole SSH lorsqu'elle est connectée sur le même réseau wifi que l'ordinateur. Le plus simple est de configurer un réseau wifi à partir d'un téléphone portable (en faisant un "partage de connexion"), tel que :

- le nom du réseau / SSID soit "PE"
- le mot de passe soit "droneautom"
- le protocole de sécurité soit WPA2

En effet, la RaspberryPi devrait s'y connecter automatiquement au démarrage. Ensuite, dans un terminal, il faut taper "ssh pi@" suivi de l'adresse IP de la RaspberryPi qui doit être donnée par le menu du téléphone portable, ou par scan du réseau avec un outil tel que NMAP (la RaspberryPi devrait apparaître sous le nom "RaspPe"). Enfin, il faut rentrer le mot de passe de l'utilisateur "pi" (auquel on se connecte, d'où le "pi@..."), à savoir "pe_drone". À ce stade, la console (au format linux) de la Raspberry Pi est accessible.

Si l'OS installé dispose d'un bureau (ce qui peut être pertinent à des fins éducatives/de prise en main), il est possible d'y accéder à distance en activant le serveur VNC sur la RaspberryPi (sudo raspi-config dans la console accessible via SSH, puis "Interface Options" puis "Enable VNC? Yes") et en installant sur son propre ordinateur le logiciel VNC viewer. Quand la RaspberryPi et l'ordinateur sont connectés au même réseau WiFi, il suffit de renseigner l'adresse IP de la RaspberryPi dans VNC Viewer puis de se connecter avec les identifiants de la RaspberryPi (pi et pe_drone) pour accéder au bureau de la RaspberryPi. Il est important de noter que la connexion VNC ne marchera pas si le port "Legacy Camera" est activé (également dans le menu Interface Options"). Il s'agit du port auquel la caméra Arducam est connecté via le câble ruban, on ne peut donc pas accéder au bureau de la RaspberryPi à distance et utiliser la caméra classique en même temps.

Les opérations qui ont été réalisés permettant la connexion à distance sont les suivantes :
Le processus d'installation de la RaspberryPi lors d'une première utilisation implique

de "flasher" le système d'exploitation voulue sur une carte SD, dans notre cas il s'agit du RaspberryPi OS Desktop. Ensuite, en ajoutant le script suivant (attention à ce que les retours chariots suivent la convention linux) :

```
country=us
update_config=1
ctrl_interface=/var/run/wpa_supplicant

5 network={
    scan_ssid=1
    ssid="PE"
    psk="droneautom"
}
```

écrit dans un fichier nommé "wpa_supplicant.conf" dans la partition "BOOT" de la carte SD, la RaspberryPi cherchera à se connecter au réseau wifi défini précédemment sans qu'on ait besoin d'accéder à l'interface avec un écran et un clavier. De la même manière, ajouter dans "BOOT" un fichier vide sans extension nommé "SSH" permettra d'autoriser la connection à distance. Ces opérations doivent être réalisés avant la première initialisation de la RaspberryPi, c'est à dire avant la mise sous tension avec la carte SD contenant l'OS insérée. Ces opérations n'auront pas a priori pas besoin d'être répétées. S'il elles devaient l'être, le mot de passe de l'utilisateur "pi" serait réinitialisé à "raspberrypi", et le nom sur le réseau de la RaspberryPi (ou hostname) à "raspberrypi".

6.5.2 Interfaces RaspberryPi - Pix Hawk 4 mini

L'alimentation de la RaspberryPi en vol sur le drone est réalisée en connectant les ports 3.3V et Gnd de la sortie ADC (pour Alimentation DC) sur le contrôleur de vol Pix Hawk 4 mini aux ports correspondant parmi les ports GPIO de la RaspberryPi (schéma 10). La RaspberryPi peut donner des ordres au contrôleur de vol et recevoir des informations de sa part à l'aide du protocole Mavlink, les 2 étant connectés par leur ports UART. Ce port n'existe pas par défaut sur la RaspberryPi, il faut le faire apparaître en reconfigurant les ports GPIO 10 de la RaspberryPi. Cela occupera les ports GPIO 14 et 15.

6.6 Programmes de reconnaissance d'image

6.6.1 Script python implémentant la reconnaissance de la cible ainsi que son positionnement

```
import cv2
import numpy as np

frameWidth= 1280
5 frameHeight= 720
cap= cv2.VideoCapture(0)
cap.set(3, frameWidth) # largeur fenetre
cap.set(4, frameHeight) #hauteur fenetre
cap.set(10, 150) #luminosité

10
couleursDetect= [[0, 123, 125, 11, 255, 255], [162, 14, 217, 179, 165,
255], [149, 36, 140, 179, 140, 255], [146, 61, 61, 179, 173, 255]]

def findColor(img):
    centroidTrouve = False
15    imgHSV = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    for couleur in couleursDetect:
        lower = np.array([couleur[0], couleur[1], couleur[2]])
        upper = np.array([couleur[3], couleur[4], couleur[5]])
        mask = cv2.inRange(imgHSV, lower, upper)
20        #cv2.imshow(str(couleur[0]), mask)
        getContours(mask)
        if centroidTrouve:
            break

25 def getContours(img):
    contours, hierarchy = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.
CHAIN_APPROX_NONE)
    for cnt in contours:
        area = cv2.contourArea(cnt)
        #print(area)
30        if area > 300:
            peri = cv2.arcLength(cnt, True)
            approx = cv2.approxPolyDP(cnt, 0.03 * peri, True)
            objCorners = len(approx)
            if objCorners == 4:
35                centroidTrouve = True
```

```

cv2.drawContours(imgResult, cnt, -1, (255, 0, 0), 3)
M = cv2.moments(cnt)

## Partie sur la correction de l'orientation du drone

# calculate x,y coordinate of center
cX = int(M["m10"] / M["m00"])
cY = int(M["m01"] / M["m00"])
cv2.circle(imgResult, (cX, cY), 5, (255, 255, 255), -1)
cv2.putText(imgResult, "centroid", (cX - 25, cY - 25), cv2.
.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2)
#print("centroid", cX, cY)
if cX>frameWidth/2-40 and cX<frameWidth/2+40 and cY>
frameHeight/2-40 and cY<frameHeight/2+40 :
    cv2.putText(imgResult, "Cible Alignee", (25, 25), cv2.
FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
    print('Cible Alignée')
    return 0

elif cX>frameWidth/2+40:
    cv2.putText(imgResult, "Tourner a droite", (25, 25),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 200), 2)
    print('Tourner à droite')
    return 1

elif cX<frameWidth/2-40:
    cv2.putText(imgResult, "Tourner a gauche", (25, 25),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 200), 2)
    print('Tourner à gauche')
    return 2

elif cY > frameHeight / 2 + 40:
    cv2.putText(imgResult, "Monter", (25, 25), cv2.
FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 200), 2)
    print('Monter')
    return 3

else:
    cv2.putText(imgResult, "Descendre", (25, 25), cv2.
FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 200), 2)
    print('Descendre')
    return 4

```

```
while True:
    success, img = cap.read()
    imgResult = img.copy()
    findColor(img)
    cv2.circle(imgResult, (frameWidth//2, frameHeight//2), 40, (0, 255, 0)
    , 2)
    cv2.imshow("Resultat", imgResult)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

codes/cible.py

6.6.2 Script Python permettant d'obtenir les paramètres propres à une couleur

```
import cv2
import numpy as np

def empty(a):
    pass

path= 'ressources/rouge.jpg'

cv2.namedWindow("TrackBars")
cv2.resizeWindow("TrackBars", 640, 240)

cv2.createTrackbar("Hue Min", "TrackBars", 16, 179, empty)
cv2.createTrackbar("Hue Max", "TrackBars", 27, 179, empty)
cv2.createTrackbar("Sat Min", "TrackBars", 162, 255, empty)
cv2.createTrackbar("Sat Max", "TrackBars", 255, 255, empty)
cv2.createTrackbar("Val Min", "TrackBars", 120, 255, empty)
cv2.createTrackbar("Val Max", "TrackBars", 190, 255, empty)

while True:
    img = cv2.imread(path)
    imgHSV = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    hmin= cv2.getTrackbarPos("Hue Min", "TrackBars")
    hmax= cv2.getTrackbarPos("Hue Max", "TrackBars")
    smin= cv2.getTrackbarPos("Sat Min", "TrackBars")
    smax= cv2.getTrackbarPos("Sat Max", "TrackBars")
```

```
30 vmin= cv2.getTrackbarPos("Val Min", "TrackBars")
    vmax= cv2.getTrackbarPos("Val Max", "TrackBars")
    print(hmin, hmax, smin, smax, vmin, vmax)
    lower= np.array([hmin, smin, vmin])
    upper = np.array([hmax, smax, vmax])
    mask= cv2.inRange(imgHSV, lower, upper)
    imgResult= cv2.bitwise_and(img, img, mask= mask)

35
    cv2.imshow("Original", img)
    cv2.imshow("HSV", imgHSV)
    cv2.imshow("Mask", mask)
    imgResize = cv2.resize(imgResult, (600,400))
40 cv2.imshow("Resultat", imgResize)
    cv2.waitKey(1)
```

codes/param_couleur.py

6.7 Programmes de mesure de distance en utilisant les télémètres à ultrasons

6.7.1 Mesure d'une distance par un capteur

```
#Libraries
import RPi.GPIO as GPIO
import time

5 #GPIO Mode (BOARD / BCM)
GPIO.setmode(GPIO.BCM)

def distance(GPIO_TRIGGER, GPIO_ECHO):
    #set GPIO direction (IN / OUT)
    10 GPIO.setup(GPIO_TRIGGER, GPIO.OUT)
    GPIO.setup(GPIO_ECHO, GPIO.IN)

    # set Trigger to HIGH
    GPIO.output(GPIO_TRIGGER, True)

    15 # set Trigger after 0.01ms to LOW
    time.sleep(0.00001)

    GPIO.output(GPIO_TRIGGER, False)

    20 StartTime = time.time()
    StopTime = time.time()

    # save StartTime
    25 while GPIO.input(GPIO_ECHO) == 0:
        StartTime = time.time()

    # save time of arrival
    while GPIO.input(GPIO_ECHO) == 1:
    30 StopTime = time.time()

    # time difference between start and arrival
    TimeElapsed = StopTime - StartTime
    # multiply with the sonic speed (34300 cm/s)
    35 # and divide by 2, because there and back
    distance = (TimeElapsed * 34300) / 2
```

```
return distance
```

codes/mesure_distance.py

6.7.2 Mesure de distance en continu

```
# importation de la fonction distance et de la librairie "time"
import time
from mesure_distance import distance

5 # On choisit les pins GPIO connectes au capteur
GPIO_TRIGGER = 18
GPIO_ECHO = 24

if __name__ == '__main__':
10     try:
        while True:
            dist = distance(GPIO_TRIGGER, GPIO_ECHO)
            print ("Measured Distance = %.1f cm" % dist)
            time.sleep(1)

15         # Reset by pressing CTRL + C
    except KeyboardInterrupt:
        print("Measurement stopped by User")
        GPIO.cleanup()
```

codes/distance_en_continu.py

6.7.3 Code pour faire plusieurs mesures à distances croissantes

```
# import de la fonction distance et des librairies utiles
from matplotlib import pyplot as plt
import RPi.GPIO as GPIO
from mesure_distance import distance

5 # On choisit les pins GPIO connectes au capteur
GPIO_TRIGGER = 18
GPIO_ECHO = 24

10 # NOTE : BIEN FAIRE LES MESURES DANS L'ORDRE
# CROISSANT DE DISTANCE ENTRE LE CAPTEUR ET
```

```
# L'OBSTACLE

if __name__ == '__main__':
    dist_reelle = []
    dist_exp = []
    try:
        while True:
            x = input("Entrer distance a l'obstacle (en cm) : ")
            dist_reelle.append(float(x))

            dist = distance(GPIO_TRIGGER, GPIO_ECHO)

            print("Distance mesuree = %.1f cm" % dist)
            dist_exp.append(dist)

            # Arrêter en pressant CTRL + C
        except KeyboardInterrupt:
            print("Arret des mesures")
            GPIO.cleanup()

            # calcul du nombre de mesures effectuees
            n = len(dist_reelle)
            nombre_essais = list(range(1, n + 1))

            # determination de la distance maximale enregistree
            maximum = max(dist_reelle + dist_exp)

            #plt.plot(nombre_essais, dist_reelle)
            plt.scatter(dist_reelle, dist_exp, label = "distance mesuree en
fonction de la distance reelle")
            #plt.xlabel("Numero de l'essai")
            plt.plot(dist_reelle, dist_reelle, label = "vraie distance")
            plt.xlabel("Distance reelle entre l'obstacle et le capteur (en cm)
")
            plt.ylabel("Distance mesuree l'obstacle et le capteur (en cm)")

            plt.axis([0, maximum, 0, maximum])
            plt.legend(loc = 'best')

            erreur_relative = []
            for i in range(n):
```

```

        erreur_relative.append(100*abs(dist_reelle[i] - dist_exp[i])/
dist_reelle[i])

plt.figure()
plt.scatter(dist_reelle, erreur_relative, label = "Ecart relatif
entre la valeur mesuree et la distance reelle (en %)")
55 plt.xlabel("Distance reelle entre l'obstacle et le capteur (en cm)
")
plt.ylabel("Ecart relatif a la valeur mesuree")

plt.axis([0, maximum, 0, max(erreur_relative)+30])
plt.legend(loc = 'best')
60

plt.show()

```

codes/Test_mesures_distances_croissantes.py

6.7.4 Code pour faire plusieurs mesures à distance constante et angle croissant

```

# import de la fonction distance et des librairies utiles
from matplotlib import pyplot as plt
import RPi.GPIO as GPIO
from mesure_distance import distance
5

# On choisit les pins GPIO connectes au capteur
GPIO_TRIGGER = 18
GPIO_ECHO = 24

10 # NOTE : BIEN FAIRE LES MESURES DANS UN
# SEUL SENS

if __name__ == '__main__':
    angle = []
    dist_exp = []
    15
    try:
        dist_reelle = float(input("Entrer la distance a l'obstacle (en cm)
: "))
        print("NOTE : cette distance doit rester la meme tout au court de
l'experience")
        while True:
            20
            x = input("Entrer l'angle entre l'axe du capteur et l'obstacle
(en degrees) : ")

```



```
        angle.append(float(x))

        dist = distance(GPIO_TRIGGER, GPIO_ECHO)

        print("Distance mesuree = %.1f cm" % dist)
        dist_exp.append(dist)

    # Arrêter en pressant CTRL + C
except KeyboardInterrupt:
    print("Arret des mesures")
    GPIO.cleanup()

    # calcul du nombre de mesures effectuees
    n = len(angle)
    nombre_essais = list(range(1, n + 1))

    # determination de la plage maximale angulaire
    minimum = min(angle)
    maximum = max(angle)

    max_distance = max(dist_exp)
    distance_reelle = n * [dist_reelle]

    plt.plot(nombre_essais, dist_reelle)
    plt.plot(angle, dist_exp, label = "distance mesuree")
    plt.xlabel("Numero de l'essai")
    plt.plot(angle, distance_reelle, label = "vraie distance")
    plt.ylabel("Distance entre l'obstacle et le capteur (en cm)")
    plt.xlabel("Angle entre l'obstacle et l'axe du capteur")

    plt.axis([minimum, maximum, -dist_reelle, max_distance*1.2])
    plt.legend(loc = 'best')

    erreur_relative = []
    for i in range(n):
        erreur_relative.append(100*abs(dist_reelle - dist_exp[i])/
dist_reelle)

    plt.figure()
    plt.scatter(angle, erreur_relative, label = "Ecart relatif entre
la valeur mesuree et la distance reelle (en %)")
    plt.xlabel("Angle entre l'obstacle et l'axe du capteur")
```

```
plt.ylabel("Ecart relatif a la valeur mesuree")

plt.axis([0, maximum, 0, max(erreur_relative)+10])
plt.legend(loc = 'best')

plt.show()
```

codes/Test_distance_en_fonction_de_l_angle.py

6.8 Check-list

	Vérification présence	Vérification qualité
Contenu		
Résumé en français	✓	✓
Résumé en anglais	✓	✓
Table des matières	✓	✓
Table des figures	✓	✓
Introduction	✓	✓
Conclusion générale	✓	✓
Bibliographie	✓	✓
Citation des références dans le texte	✓	✓
Forme		
Vérification orthographe	✓	✓
Pagination	✓	✓
Homogénéité de la mise en page	✓	✓
Lisibilité des figures	✓	✓